ENOKON

Documentazione del caso di studio

Wine Knowledge-Based Quality Classification

Gruppo di lavoro:

- Pietragalla Federico – 808958
- f.pietragalla@studenti.uniba.it

AA 2025-2026

link al repository

INTRODUZIONE

Per lo sviluppo del presente progetto è stato utilizzato il linguaggio di programmazione **Python**, con l'obiettivo di modellare e analizzare il problema della **classificazione della qualità sensoriale del vino** a partire dalle sue misure fisico-chimiche. Il dominio considerato riguarda la previsione della classe di qualità di un vino — *bassa*, *media* o *alta* — stabilita dal giudizio di degustatori esperti, integrando tecniche di apprendimento automatico, rappresentazione formale della conoscenza mediante un'ontologia OWL 2 e ragionamento simbolico tramite una Knowledge Base Prolog.

Il dataset impiegato è **Wine Quality** dell'UCI Machine Learning Repository (Cortez et al., 2009) e riguarda vini portoghesi *vinho verde*: **1599 rossi e 4898 bianchi**, per un totale di 6497 record. Ogni vino è descritto da 11 misure fisico-chimiche e da un punteggio di qualità, che è la mediana di almeno tre valutazioni sensoriali di esperti su una scala da 0 a 10 (nel dataset i valori vanno da 3 a 9). I due file originali vengono scaricati e uniti dallo script `data/download_dataset.py`, che aggiunge la colonna categoriale `wine_type`. Dopo la rimozione dei record duplicati restano **5320 vini** (1359 rossi e 3961 bianchi).

Struttura del dataset

Le feature del dataset sono le seguenti:

- **fixed_acidity**: acidità fissa in g/L (espressa come acido tartarico), cioè gli acidi non volatili che danno freschezza e struttura al vino; nel dataset varia tra 3.8 e 15.9 g/L.
- **volatile_acidity**: acidità volatile in g/L (come acido acetico); in eccesso conferisce note di aceto ed è considerata un difetto (0.08–1.58 g/L).
- **citric_acid**: acido citrico in g/L; in piccole quantità aggiunge freschezza (0.00–1.66 g/L).
- **residual_sugar**: zuccheri residui in g/L, non trasformati in alcol dalla fermentazione; determinano la dolcezza del vino (0.6–65.8 g/L).
- **chlorides**: cloruri in g/L (come cloruro di sodio), indice del contenuto salino (0.009–0.611 g/L).
- **free_sulfur_dioxide**: anidride solforosa (SO_2) libera in mg/L, la frazione attiva con azione antiossidante e antimicrobica (1–289 mg/L).
- **total_sulfur_dioxide**: SO_2 totale (libera + legata) in mg/L, soggetta a limiti di legge (6–440 mg/L).
- **density**: densità in g/cm^3; diminuisce al crescere dell'alcol e aumenta con gli zuccheri (0.987–1.039).
- **pH**: acidità reale su scala logaritmica; valori alti riducono l'efficacia della solforosa (2.72–4.01).
- **sulphates**: solfati in g/L (come solfato di potassio), additivo con azione antimicrobica e antiossidante (0.22–2.00 g/L).
- **alcohol**: grado alcolico in % vol (8.0–14.9 % vol).

A queste si aggiunge la variabile categoriale **wine_type** (`red` / `white`), trasformata in `tipo` (*rosso* / *bianco*) per l'ontologia e la KB e nell'indicatore numerico `is_red` per i modelli.

La variabile target è:

- **quality**: punteggio sensoriale intero, discretizzato in tre classi ordinali (`LOW_MAX = 5` e `HIGH_MIN = 7` in `config.py`). I punteggi estremi sono rari (30 vini con punteggio 3, 206 con 4, 148 con 8 e solo 5 con 9) e darebbero classi quasi vuote: per questo si lavora con tre classi.

Indice	Classe	Punteggi	N	Percentuale	Rossi	Bianchi
0	bassa	≤ 5	1988	37.4%	640	1348
1	media	6	2323	43.7%	535	1788
2	alta	≥ 7	1009	19.0%	184	825

Il dataset presenta uno **sbilanciamento di classe**: la classe *alta* (19.0%) è meno della metà della classe *media* (43.7%). Inoltre rossi e bianchi hanno distribuzioni diverse: è di qualità bassa il 47.1% dei rossi contro il 34.0% dei bianchi, e di qualità alta il 13.5% dei rossi contro il 20.8% dei bianchi. Questo ha motivato l'adozione di **metriche macro** e della **G-Mean** (media geometrica delle recall per classe) accanto alla F1 macro, poiché la G-Mean penalizza i modelli che classificano bene solo le classi maggioritarie, e l'uso di **pesi di classe bilanciati** nei modelli supervisionati.

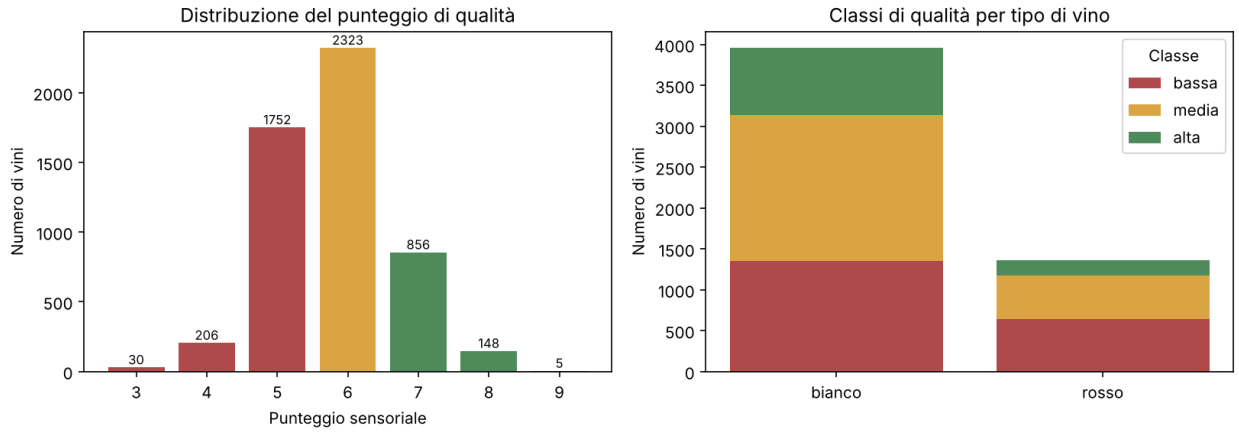


Figura 1 — Distribuzione del punteggio sensoriale (a sinistra) e delle tre classi di qualità per tipo di vino (a destra), dopo la rimozione dei duplicati. Colori: rosso = bassa, giallo = media, verde = alta.

Obiettivi del progetto

Gli obiettivi del progetto sono:

- predire la **classe di qualità** di un vino attraverso modelli di apprendimento supervisionato e semi-supervisionato, confrontando le prestazioni con e senza l'integrazione di Background Knowledge ontologica;
- costruire un'**ontologia OWL 2** (`wine.owl`) che formalizzi la conoscenza enologica, definendo concetti derivati dalle misure chimiche tramite restrizioni sulle data property, e usarla per arricchire la rappresentazione delle istanze;
- rappresentare la **conoscenza simbolica** del dominio mediante una Knowledge Base Prolog (`wine_kb.pl`) con regole di derivazione dei concetti, regole di classificazione da sistema esperto e spiegazione della decisione;
- modellare le **dipendenze probabilistiche** tra i concetti tramite una rete bayesiana discreta (Tree-Augmented Naive Bayes), effettuando inferenza esatta sulla classe di qualità data l'evidenza;
- confrontare in modo **statisticamente fondato** tutti i modelli (test di Friedman e post-hoc di Nemenyi).

Le domande di ricerca che guidano la sperimentazione sono:

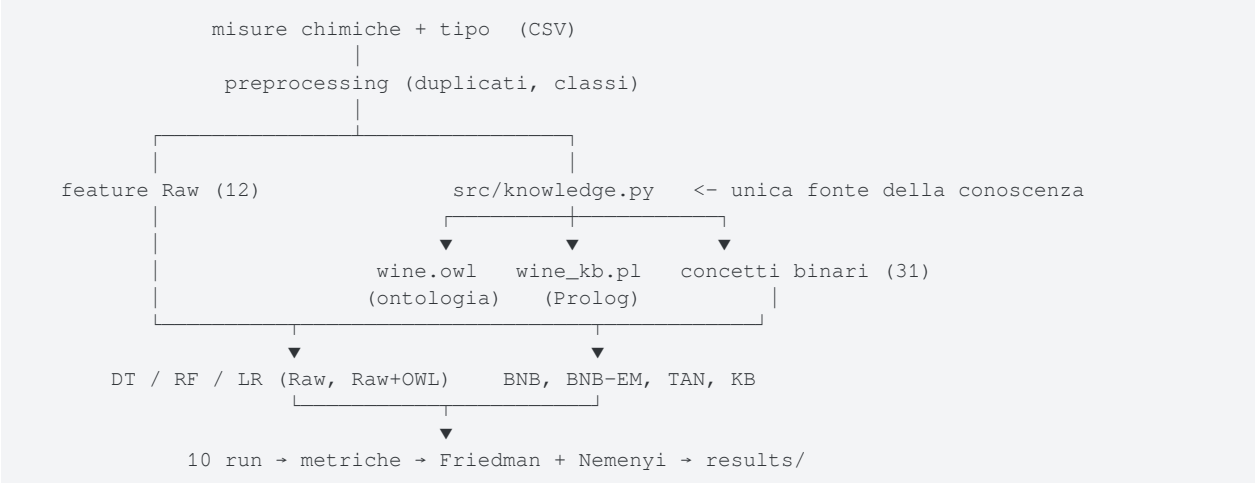
- **RQ1** — I concetti enologici dell'ontologia, usati come feature aggiuntive, migliorano i classificatori classici?

- **RQ2** — Un classificatore generativo (Bernoulli Naive Bayes) trae vantaggio dai dati non etichettati tramite EM?
- **RQ3** — La rete bayesiana TAN supera il Naive Bayes modellando le dipendenze tra i concetti?
- **RQ4** — Quanto si avvicina ai modelli appresi un sistema esperto fatto solo di regole (KB Prolog), che non vede mai le etichette?

Architettura del sistema

La scelta architetturale principale è che **la conoscenza di dominio è definita una sola volta**, nel modulo `src/knowledge.py`: da lì vengono derivati automaticamente, senza duplicazioni a mano, l'ontologia OWL, la Knowledge Base Prolog e le feature binarie usate dai modelli. Per cambiare una soglia basta modificare quel file: `main.py` rigenera ontologia e KB a ogni esecuzione e `tests/test_consistency.py` verifica che OWL e Prolog esprimano esattamente ciò che esprime Python.

Architettura complessiva della pipeline (main.py)



Modulo	Ruolo
`src/knowledge.py`	**Unica fonte** della conoscenza: proprietà, concetti di soglia, di tipo e composti, regole di classificazione; valutazione vettoriale dei concetti e delle regole
`ontology/build_ontology.py`	Genera `wine.owl` (OWL 2, RDF/XML) dalla conoscenza
`kb/build_kb.py`	Genera `wine_kb.pl` (Prolog) dalla conoscenza
`src/preprocessing.py`	Caricamento, pulizia, classi, split stratificati, sottoinsieme etichettato
`src/ontology_features.py`	Concetti → feature binarie, copertura per classe, catalogo dei concetti
`src/supervised.py`	Decision Tree, Random Forest, Logistic Regression (scikit-learn)
`src/bernoulli_nb.py`, `src/bnb_em.py`	Bernoulli Naive Bayes e BNB semi-supervisionato con EM (numpy)
`src/bayesian_network.py`	Rete bayesiana Tree-Augmented Naive Bayes (numpy)
`src/prolog_kb.py`	Esecuzione della KB: SWI-Prolog oppure motore Python equivalente
`src/evaluation.py`, `src/visualization.py`	Metriche, test di Friedman e Nemenyi, grafici
`tests/test_consistency.py`	Coerenza OWL/Prolog/Python e correttezza dei modelli (10 test)

ELENCO DI ARGOMENTI DI INTERESSE

Gli argomenti oggetto del corso di Ingegneria della Conoscenza che sono stati applicati nel presente progetto sono i seguenti.

Background Knowledge Ontologica (ML+OntoBK):

È stata costruita un'ontologia OWL 2 (`wine.owl`) contenente 52 classi (37 definite tramite equivalenza), 13 data property funzionali e 12 assiomi di disgiunzione. I 34 concetti enologici definiti (26 di soglia, 2 di tipo e 6 composti) formalizzano le misure chimiche come restrizioni su intervalli di valori o come combinazioni AND/OR di altri concetti; 31 di essi diventano feature binarie di Background Knowledge per ogni vino. Questo approccio permette di confrontare sistematicamente l'impatto della conoscenza ontologica sull'apprendimento.

Apprendimento supervisionato:

Sono state adottate tre tecniche di machine learning per il problema di classificazione multiclasse a 3 classi: **Decision Tree (DT)**, **Random Forest (RF)** e **Logistic Regression (LR)**, tutte con pesi di classe bilanciati. I modelli sono stati addestrati e confrontati sia sulle feature grezze (*Raw*: 11 misure + tipo) sia su quelle combinate con la Background Knowledge OWL (*Raw+OWL*). Gli iperparametri sono fissati in `config.py`; con l'opzione `--tune` vengono selezionati tramite **GridSearchCV** con cross-validation stratificata interna al solo training set. La valutazione avviene tramite **StratifiedShuffleSplit** su 10 run indipendenti (70/30), riportando media e deviazione standard di Precision, Recall, F1 e G-Mean, e le differenze tra i modelli sono verificate con i test di **Friedman** e **Nemenyi**.

Apprendimento semi-supervisionato (BNB e BNB-EM):

Sono stati implementati da zero (numpy) il **Bernoulli Naive Bayes** (BNB) per le feature binarie OWL e la sua versione semi-supervisionata **BNB-EM**, che applica l'algoritmo Expectation-Maximization (Nigam et al., 2000) per sfruttare anche gli esempi non etichettati: in ogni run solo il 20% del training mantiene l'etichetta. Una curva di apprendimento confronta i due modelli al variare della quota di etichette (dall'1% al 100%) e del peso λ attribuito ai dati non etichettati.

Rappresentazione e ragionamento simbolico (KB Prolog):

È stata sviluppata una **Knowledge Base Prolog** (`wine_kb.pl`), generata automaticamente dalla stessa definizione della conoscenza usata per l'ontologia, che contiene le regole di derivazione dei 34 concetti, 4 regole di classificazione da sistema esperto con priorità e un predicato di spiegazione `classifica(Vino, Classe, Motivo)`. La KB viene eseguita in SWI-Prolog, se disponibile, oppure da un motore Python equivalente, e **non vede mai le etichette**.

Ragionamento su conoscenza incerta (Rete Bayesiana):

È stata costruita una **rete bayesiana discreta** con struttura **Tree-Augmented Naive Bayes** (TAN, Friedman et al., 1997) sui 31 concetti OWL: la classe è genitore di ogni concetto e i concetti formano in più un albero appreso dai dati massimizzando l'informazione mutua condizionata (algoritmo di Prim). Le tabelle di probabilità condizionata sono stimate con smoothing di Laplace e l'inferenza di P(Qualità | concetti) è esatta.

Background Knowledge Ontologica

Introduzione e motivazione

Nel contesto del progetto ENOKON, la qualità di un vino non dipende dalle singole misure chimiche considerate in astratto, ma dalla loro **interpretazione enologica**. Valori come `volatile_acidity = 0.70` g/L o `alcohol = 12.8` % vol assumono significato solo se contestualizzati: oltre circa 0.6 g/L l'acido acetico diventa percepibile come spunto di aceto (un difetto), mentre un grado alcolico alto senza spunto acetico indica uve mature e una fermentazione pulita. Alcune nozioni dipendono inoltre dalla tipologia: il limite di legge sull'anidride solforosa totale è diverso per rossi e bianchi.

L'approccio **ML+OntoBK** (Machine Learning con Background Knowledge Ontologica) formalizza questa conoscenza di dominio in un'ontologia OWL, dalla quale vengono estratte feature booleane derivate che arricchiscono la rappresentazione delle istanze. Questo approccio è ispirato alla categoria progettuale indicata nelle linee-guida ICon: *sviluppare e testare un sistema basato su modelli di apprendimento supervisionato che integri i dataset con conoscenza di fondo proveniente dal Web Semantico*.

La conoscenza è organizzata in tre livelli: **concetti di soglia**, che discretizzano una misura in un livello qualitativo come farebbe un enologo (`AlcolAlto`, `PHBasso`, `Secco`, ...); **concetti composti**, nozioni enologiche di livello superiore costruite con AND/OR (`VinoDifettoso`, `VinoEquilibrato`, `RischioInstabilita`, ...); **regole di classificazione**, che formano il sistema esperto della KB Prolog.

Struttura dell'ontologia OWL (wine.owl)

L'ontologia è generata dallo script `ontology/build_ontology.py` in formato RDF/XML standard (OWL 2), usando solo la libreria standard di Python (`xml.etree`), ed è apribile con Protégé o owlready2. Contiene **52 classi** (37 definite tramite `owl:equivalentClass`), **13 data property funzionali** e **12 assiomi di disgiunzione** (`owl:AllDisjointClasses`). È strutturata nei livelli seguenti:

Classe radice e classi di raggruppamento

- **Vino**: classe radice dell'ontologia. Ogni individuo corrisponde a un campione di vino descritto dalle sue misure chimiche e dal punteggio sensoriale.
- **13 classi-famiglia**, sottoclassi di `Vino`, che raggruppano i concetti affini: `TipoVino`, `ProfiloAciditaFissa`, `ProfiloAciditaVolatile`, `ProfiloAcidoCitrico`, `ProfiloZuccherino`, `ProfiloCloruri`, `ProfiloSO2Libera`, `ProfiloSO2Totale`, `ProfiloDensita`, `ProfiloPH`, `ProfiloSolfati`, `ProfiloAlcolico`, `ProfiloEnologico`; a queste si aggiunge `QualitaVino`, superclasse delle tre classi bersaglio.

Data Properties (misure chimiche)

Definite sul dominio `Vino` e tutte funzionali (`owl:FunctionalProperty`): una per ciascuna misura chimica, più il tipo di vino e il punteggio sensoriale.

Data Property	Tipo	Significato enologico
aciditaFissa	xsd:decimal (g/L)	Acidi non volatili (come acido tartarico)
aciditaVolatile	xsd:decimal (g/L)	Acido acetico: oltre soglia è un difetto
acidoCitrico	xsd:decimal (g/L)	Freschezza aromatica
zuccheriResidui	xsd:decimal (g/L)	Dolcezza del vino

Data Property	Tipo	Significato enologico
cloruri	xsd:decimal (g/L)	Contenuto salino
so2Libera	xsd:decimal (mg/L)	Solforosa attiva (protezione antiossidante e antimicrobica)
so2Totale	xsd:decimal (mg/L)	Solforosa libera + legata (soggetta a limiti di legge)
densita	xsd:decimal (g/cm^3)	Dipende da alcol (la abbassa) e zuccheri (la alzano)
ph	xsd:decimal	Acidità reale (scala logaritmica)
solfati	xsd:decimal (g/L)	Additivo antimicrobico e antiossidante
gradoAlcolico	xsd:decimal (% vol)	Maturità delle uve e struttura del vino
tipoVino	xsd:string	"rosso" oppure "bianco"
punteggioQualita	xsd:integer	Punteggio sensoriale (mediana di 3+ degustatori)

Concetti derivati di soglia (26) e di tipo (2)

Ogni concetto di soglia è una **classe definita** (condizioni necessarie e sufficienti) tramite `owl:equivalentClass`: l'intersezione di `Vino` con una restrizione esistenziale sulla data property, il cui range è un datatype `xsd:decimal` ristretto con i facet `minInclusive`, `minExclusive`, `maxInclusive` e `maxExclusive`. I due concetti di tipo usano invece una restrizione `owl:hasValue`. Gli intervalli seguono la convenzione [lo, hi) salvo diversa indicazione:

wine.owl — Concetti di soglia e di tipo (notazione DL)

```
VinoRosso              ≡ Vino ⊓ ∃tipoVino.{"rosso"}
VinoBianco             ≡ Vino ⊓ ∃tipoVino.{"bianco"}
AciditaFissaBassa      ≡ Vino ⊓ ∃aciditaFissa.(xsd:decimal[< 6.0])
AciditaFissaAlta       ≡ Vino ⊓ ∃aciditaFissa.(xsd:decimal[≥ 8.5])
AciditaVolatileBassa   ≡ Vino ⊓ ∃aciditaVolatile.(xsd:decimal[< 0.3])
AciditaVolatileElevata ≡ Vino ⊓ ∃aciditaVolatile.(xsd:decimal[≥ 0.6])
AcidoCitricoBasso      ≡ Vino ⊓ ∃acidoCitrico.(xsd:decimal[< 0.15])
AcidoCitricoElevato    ≡ Vino ⊓ ∃acidoCitrico.(xsd:decimal[≥ 0.5])
Secco                  ≡ Vino ⊓ ∃zuccheriResidui.(xsd:decimal[≤ 4.0])
Abboccato              ≡ Vino ⊓ ∃zuccheriResidui.(xsd:decimal[> 4.0, ≤ 12.0])
Amabile                ≡ Vino ⊓ ∃zuccheriResidui.(xsd:decimal[> 12.0])
CloruriBassi           ≡ Vino ⊓ ∃cloruri.(xsd:decimal[< 0.05])
CloruriElevati         ≡ Vino ⊓ ∃cloruri.(xsd:decimal[≥ 0.1])
SO2LiberaBassa         ≡ Vino ⊓ ∃so2Libera.(xsd:decimal[< 15.0])
SO2LiberaAdeguata      ≡ Vino ⊓ ∃so2Libera.(xsd:decimal[≥ 15.0, ≤ 50.0])
SO2LiberaAlta          ≡ Vino ⊓ ∃so2Libera.(xsd:decimal[> 50.0])
SO2TotaleOltre150      ≡ Vino ⊓ ∃so2Totale.(xsd:decimal[> 150.0])      (ausiliario)
SO2TotaleOltre200      ≡ Vino ⊓ ∃so2Totale.(xsd:decimal[> 200.0])      (ausiliario)
DensitaBassa           ≡ Vino ⊓ ∃densita.(xsd:decimal[< 0.993])
DensitaAlta            ≡ Vino ⊓ ∃densita.(xsd:decimal[≥ 0.997])
PHBasso                ≡ Vino ⊓ ∃ph.(xsd:decimal[< 3.2])
PHMedio                ≡ Vino ⊓ ∃ph.(xsd:decimal[≥ 3.2, < 3.5])
PHAlto                 ≡ Vino ⊓ ∃ph.(xsd:decimal[≥ 3.5])
SolfatiBassi           ≡ Vino ⊓ ∃solfati.(xsd:decimal[< 0.5])
SolfatiAlti            ≡ Vino ⊓ ∃solfati.(xsd:decimal[≥ 0.75])
AlcolBasso             ≡ Vino ⊓ ∃gradoAlcolico.(xsd:decimal[< 10.0])
AlcolMedio             ≡ Vino ⊓ ∃gradoAlcolico.(xsd:decimal[≥ 10.0, < 12.0])
AlcolAlto              ≡ Vino ⊓ ∃gradoAlcolico.(xsd:decimal[≥ 12.0])
```

I concetti mutuamente esclusivi di una stessa famiglia (ad esempio `AlcolBasso`, `AlcolMedio` e `AlcolAlto`) sono dichiarati in un assioma `AllDisjointClasses`: 11 assiomi per le famiglie di concetti più uno per le classi di qualità.

Concetti derivati composti (6) — intersezioni e unioni OWL

I concetti composti sono definiti come **intersezioni** (⊓, `owl:intersectionOf`) e **unioni** (⊔, `owl:unionOf`) di altri concetti. Corrispondono a ciò che un reasoner OWL (HermiT/Pellet) dedurrebbe automaticamente classificando un individuo `Vino` in base alle sue misure, e rappresentano nozioni enologiche di livello superiore:

wine.owl — Concetti composti

```
SO2TotaleEccessiva ≡ (VinoRosso ⊓ SO2TotaleOltre150) ⊔ (VinoBianco ⊓ SO2TotaleOltre200)
RischioInstabilita ≡ PHAlto ⊓ SO2LiberaBassa
VinoDifettoso      ≡ AciditaVolatileElevata ⊔ SO2TotaleEccessiva ⊔ RischioInstabilita
VinoFresco         ≡ PHBasso ⊓ (AlcolBasso ⊔ AlcolMedio) ⊓ AciditaVolatileBassa
VinoEquilibrato    ≡ AlcolMedio ⊓ AciditaVolatileBassa ⊓ SO2LiberaAdeguata ⊓ (PHBasso ⊔ PHMedio)
VinoMaturoEPulito  ≡ AlcolAlto ⊓ AciditaVolatileBassa
```

Concetto	Significato enologico
SO2TotaleEccessiva	SO_2 totale oltre il limite UE per vini secchi: 150 mg/L (rossi) e 200 mg/L (bianchi)
RischioInstabilita	pH alto con SO_2 libera bassa: il vino è poco protetto, perché l'efficacia della solforosa cala al crescere del pH
VinoDifettoso	Almeno un difetto analitico: spunto acetico, eccesso di solforosa o rischio di instabilità
VinoFresco	Alta acidità, alcol contenuto, nessuno spunto acetico: profilo tipico del vinho verde
VinoEquilibrato	Alcol medio, acidità volatile bassa, protezione solforosa adeguata e pH usuale
VinoMaturoEPulito	Grado alcolico alto (uve mature) senza spunto acetico (fermentazione pulita)

Classi di qualità (bersaglio)

Le tre classi bersaglio sono anch'esse classi definite, sul punteggio sensoriale intero, e sono dichiarate mutuamente disgiunte:

wine.owl — Classi di qualità

```
QualitaBassa ≡ Vino ⊓ ∃punteggioQualita.(xsd:integer[≤ 5])
QualitaMedia ≡ Vino ⊓ ∃punteggioQualita.(xsd:integer[≥ 6, ≤ 6])
QualitaAlta  ≡ Vino ⊓ ∃punteggioQualita.(xsd:integer[≥ 7])
```

Di seguito un concetto così come appare nel file generato (estratto, IRI abbreviati):

ontology/wine.owl — Classe AlcolMedio

```
<owl:Class rdf:about="#AlcolMedio">
  <rdfs:label xml:lang="it">Vino di corpo medio.</rdfs:label>
  <rdfs:comment xml:lang="it">grado alcolico ∈ [10.0, 12.0) % vol.
      Convenzione: tra 10 e 12 % vol.</rdfs:comment>
  <rdfs:subClassOf rdf:resource="#ProfiloAlcolico" />
  <owl:equivalentClass>
    <owl:Class>
      <owl:intersectionOf rdf:parseType="Collection">
        <owl:Class rdf:about="#Vino" />
        <owl:Restriction>
          <owl:onProperty rdf:resource="#gradoAlcolico" />
          <owl:someValuesFrom>
            <rdfs:Datatype>
              <owl:onDatatype rdf:resource="&xsd;decimal" />
              <owl:withRestrictions rdf:parseType="Collection">
                <rdf:Description>
                  <xsd:minInclusive rdf:datatype="&xsd;decimal">10.0</xsd:minInclusive>
                </rdf:Description>
                <rdf:Description>
                  <xsd:maxExclusive rdf:datatype="&xsd;decimal">12.0</xsd:maxExclusive>
                </rdf:Description>
```

```
            </owl:withRestrictions>
          </rdfs:Datatype>
        </owl:someValuesFrom>
      </owl:Restriction>
    </owl:intersectionOf>
  </owl:Class>
</owl:equivalentClass>
</owl:Class>
```

Nota Metodologica: Nella pipeline l'appartenenza di ogni vino ai concetti non è calcolata da un reasoner OWL, ma dalla funzione `evaluate_concepts` di `src/knowledge.py`, che applica le stesse definizioni come predicati vettoriali (numpy) sulle colonne del DataFrame. Questo approccio è formalmente equivalente, poiché tutte le definizioni sono congiunzioni e disgiunzioni di vincoli su singole data property funzionali. L'equivalenza non è solo dichiarata ma **verificata**: `tests/test_consistency.py` contiene un interprete minimale indipendente che legge SOLO `wine.owl` e controlla, su 400 vini casuali e su 180 vini con una misura esattamente sulla soglia (o subito sotto e sopra), che OWL e Python diano lo stesso risultato per tutti i 34 concetti. La verifica con un reasoner completo (HermiT o Pellet) si può fare aprendo `wine.owl` in Protégé.

Scelta delle soglie

Le soglie sono fissate **a priori** dalla conoscenza di dominio, senza ottimizzarle sulle etichette né sulle partizioni di test, e sono di tre tipi: **normative o sensoriali** (indicazioni di dolcezza UE, limiti UE sull'anidride solforosa totale, soglie di percezione dell'acidità volatile e della SO_2 libera), **convenzioni enologiche** (pH, grado alcolico, SO_2 libera insufficiente) ed **empiriche** (acidità fissa, acido citrico, cloruri, densità, solfati). La tabella riporta per ogni famiglia la soglia, la quota di vini del dataset che soddisfa il concetto e il fondamento della scelta.

Concetto	Soglia	Copertura dataset	Fondamento
AciditaVolatileElevata	≥ 0.6 g/L	10.1%	Sensoriale: lo spunto acetico diventa percepibile
AciditaVolatileBassa	< 0.3 g/L	49.8%	Ben al di sotto della soglia di percezione
Secco / Abboccato / Amabile	≤ 4 / (4, 12] / > 12 g/L	58.0% / 31.9% / 10.2%	Normativa: indicazioni di dolcezza UE (semplificate)
SO2LiberaBassa / Adeguata / Alta	< 15 / [15, 50] / > 50 mg/L	20.6% / 66.1% / 13.2%	Convenzione (protezione insufficiente) e soglia olfattiva (~50 mg/L)
SO2TotaleEccessiva	> 150 (rossi) / > 200 (bianchi) mg/L	6.1%	Normativa: limiti UE per vini secchi (semplificati)
PHBasso / Medio / Alto	< 3.2 / [3.2, 3.5) / ≥ 3.5	45.4% / 49.2% / 5.5%	Convenzione enologica
AlcolBasso / Medio / Alto	< 10 / [10, 12) / ≥ 12 % vol	37.6% / 46.4% / 15.9%	Convenzione enologica
DensitaBassa / Alta	< 0.993 / ≥ 0.997 g/cm^3	32.6% / 22.5%	Empirica
CloruriBassi / Elevati	< 0.05 / ≥ 0.1 g/L	55.8% / 5.8%	Empirica
SolfatiBassi / Alti	< 0.5 / ≥ 0.75 g/L	44.6% / 8.0%	Empirica
AciditaFissaBassa / Alta	< 6.0 / ≥ 8.5 g/L	10.6% / 12.8%	Empirica
AcidoCitricoBasso / Elevato	< 0.15 / ≥ 0.5 g/L	10.9% / 8.6%	Empirica

A differenza di soglie calibrate sui quantili del dataset, soglie fissate a priori producono coperture molto diverse: si va da concetti rari come `RischioInstabilita` (1.5% dei vini) e `PHAlto` (5.5%) a concetti molto frequenti come `SO2LiberaAdeguata` (66.1%). È il prezzo da pagare per avere concetti con un significato

enologico preciso e nessun parametro stimato dai dati (quindi nessun rischio di data leakage).

Estrazione delle feature OWL in Python

I concetti sono dichiarati in `src/knowledge.py` con tre costruttori (`_rng` per le soglie, `_typ` per il tipo, `_cmp` per i composti) e valutati da `evaluate_concepts` nell'ordine di definizione: un concetto composto può riferirsi solo a concetti già definiti, vincolo controllato all'import del modulo. Il risultato è una matrice booleana con una colonna per concetto:

src/knowledge.py — definizione e valutazione dei concetti (estratto)

```
# Estratto da src/knowledge.py
    _rng("AlcolAlto", "alcol_alto", "ProfiloAlcolico", "alcohol",
         "Vino alcolico.", "Convenzione: da 12 % vol, indice di uve molto mature.", lo=12.0),
    ...
    _cmp("VinoMaturoEPulito", "vino_maturo_e_pulito", AND("AlcolAlto", "AciditaVolatileBassa"),
         "Vino da uve mature e ben vinificato.",
         "Grado alcolico alto (maturità delle uve) senza spunto acetico."),

def evaluate_concepts(df: pd.DataFrame) -> pd.DataFrame:
    values: Dict[str, np.ndarray] = {}
    n = len(df)
    for c in CONCEPTS:
        if c.kind == RANGE:
            x = df[c.column].to_numpy(dtype=float)
            m = np.ones(n, dtype=bool)
            if c.lo is not None:
                m &= (x >= c.lo) if c.lo_incl else (x > c.lo)
            if c.hi is not None:
                m &= (x <= c.hi) if c.hi_incl else (x < c.hi)
        elif c.kind == TYPE:
            m = np.asarray(df[TIPO_COLUMN].to_numpy() == c.value)
        else:
            m = _eval_expr(c.expr, values)        # AND/OR di concetti già valutati
        values[c.name] = np.asarray(m, dtype=bool)
    return pd.DataFrame(values, index=df.index)
```

Dei 34 concetti, 31 diventano feature binarie (`owl_feature_matrix` in `src/ontology_features.py`). Restano esclusi `VinoBianco`, complementare di `VinoRosso`, e i due concetti ausiliari `SO2TotaleOltre150` e `SO2TotaleOltre200`, che servono solo a definire `SO2TotaleEccessiva`.

Coverage dei concetti OWL per classe

La tabella seguente mostra la percentuale di vini di ciascuna classe che soddisfano i concetti più significativi. Un'alta varianza tra le colonne indica un elevato potere discriminante del concetto.

Concetto	bassa	media	alta	tutti i vini
AlcolBasso	60.6%	30.8%	8.0%	37.6%
AlcolAlto	2.9%	15.8%	42.0%	15.9%
DensitaBassa	15.7%	35.0%	60.2%	32.6%
CloruriBassi	41.4%	59.5%	75.4%	55.8%
VinoDifettoso	26.7%	13.3%	3.7%	16.5%
AciditaVolatileElevata	18.1%	6.7%	2.1%	10.1%
VinoMaturoEPulito	0.9%	7.3%	15.5%	6.5%
VinoEquilibrato	9.5%	24.4%	24.8%	18.9%

Concetto	bassa	media	alta	tutti i vini
SolfatiAlti	5.2%	8.2%	13.4%	8.1%
VinoRosso	32.2%	23.0%	18.2%	25.6%
Secco	57.1%	57.1%	61.5%	58.0%
PHAlto	6.0%	5.0%	5.5%	5.5%

`AlcolBasso` è il concetto più discriminante: lo soddisfa il 60.6% dei vini di qualità bassa contro l'8.0% di quelli di qualità alta; `AlcolAlto` ha l'andamento opposto (2.9% → 42.0%). È coerente con il fatto che il grado alcolico è la misura più correlata alla qualità. `DensitaBassa` (15.7% → 60.2%) porta un'informazione simile, perché l'alcol abbassa la densità. Il concetto composto `VinoDifettoso` cattura bene la nozione di difetto: lo soddisfa il 26.7% dei vini di qualità bassa e solo il 3.7% di quelli di qualità alta. Concetti come `Secco` e `PHAlto`, invece, hanno coperture quasi uguali nelle tre classi e da soli non discriminano.

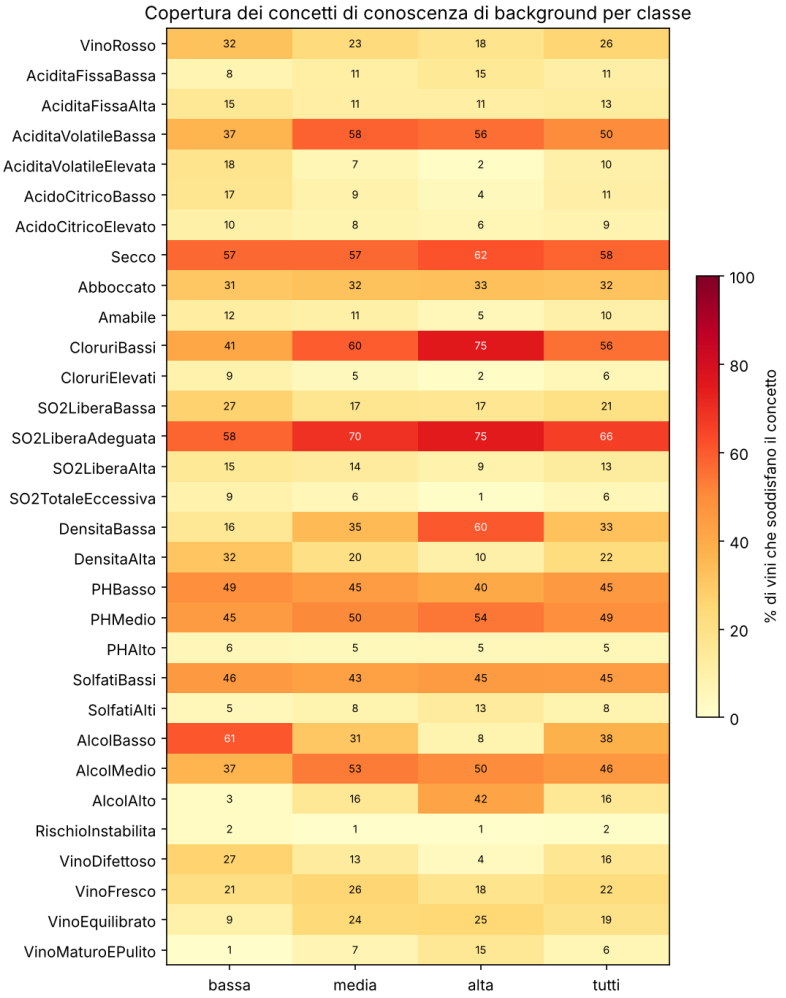


Figura 2 — Copertura dei 31 concetti di Background Knowledge per classe: percentuale di vini di ciascuna classe (e dell'intero dataset) che soddisfano il concetto. Colori più scuri indicano copertura più alta; alta varianza tra le colonne = alta discriminabilità.

Preprocessing e Feature Engineering

Caricamento e pulizia dei dati

Il dataset è caricato dal file `data/wine_quality.csv` tramite la libreria **pandas** (funzione `load_dataset` di `src/preprocessing.py`). Prima di procedere all'addestramento è stata effettuata una fase di validazione e preparazione dei dati. In particolare:

- è verificata la presenza di tutte le colonne richieste (le 11 misure, `wine_type` e `quality`) e sono scartate le eventuali righe con valori mancanti (nessuna nel dataset UCI);
- sono rimossi i **record duplicati**: 1177 su 6497 (240 tra i rossi e 937 tra i bianchi), lasciando 5320 vini;
- il tipo di vino è validato (sono ammessi solo `red` e `white`) e trasformato nella colonna `tipo` (*rosso*/*bianco*), usata da ontologia e KB, e nell'indicatore numerico `is_red`;
- il punteggio `quality` è convertito nella classe bersaglio (0 = bassa, 1 = media, 2 = alta) e nella sua etichetta leggibile `quality_label`.

```
# src/preprocessing.py — estratto
def quality_to_class(quality) -> np.ndarray:
    """Punteggio sensoriale -> indice di classe (0 = bassa, 1 = media, 2 = alta)."""
    q = np.asarray(quality)
    return np.where(q <= cfg.LOW_MAX, 0, np.where(q >= cfg.HIGH_MIN, 2, 1)).astype(int)

# in load_dataset(...):
    df = df[needed].dropna()
    n_loaded = len(df)
    if drop_duplicates:                              # config.DROP_DUPLICATES = True
        df = df.drop_duplicates()
    df = df.reset_index(drop=True)
    ...
    df[cfg.TIPO_COLUMN] = df[cfg.TYPE_COLUMN].map({"red": "rosso", "white": "bianco"})
    df["is_red"] = (df[cfg.TYPE_COLUMN] == "red").astype(int)
    df["quality_class"] = quality_to_class(df[cfg.QUALITY_COLUMN].to_numpy())
```

Nota Metodologica: Il dataset UCI contiene molti record identici, un fatto documentato in letteratura. Con split casuali lo stesso vino finirebbe sia nel training sia nel test e il modello verrebbe valutato su esempi già visti, gonfiando le metriche. Per questo i duplicati sono rimossi **prima** della costruzione degli split (`DROP_DUPLICATES = True`): il 18.1% dei record originali.

Analisi esplorativa: correlazioni con la qualità

Tra le misure, la più correlata alla qualità è il grado alcolico (r = +0.47), seguito da densità (r = −0.33), acidità volatile (r = −0.27) e cloruri (r = −0.20); tutte le altre misure hanno |r| ≤ 0.10, quindi un legame lineare debole. Le misure sono invece fortemente correlate tra loro e con il tipo di vino: SO_2 libera e totale (r = +0.72), densità e alcol (r = −0.67), tipo rosso e SO_2 totale (r = −0.69), tipo rosso e acidità volatile (r = +0.65). Rossi e bianchi formano quindi due gruppi chimicamente ben separati: un aspetto che tornerà nell'analisi del BNB-EM.

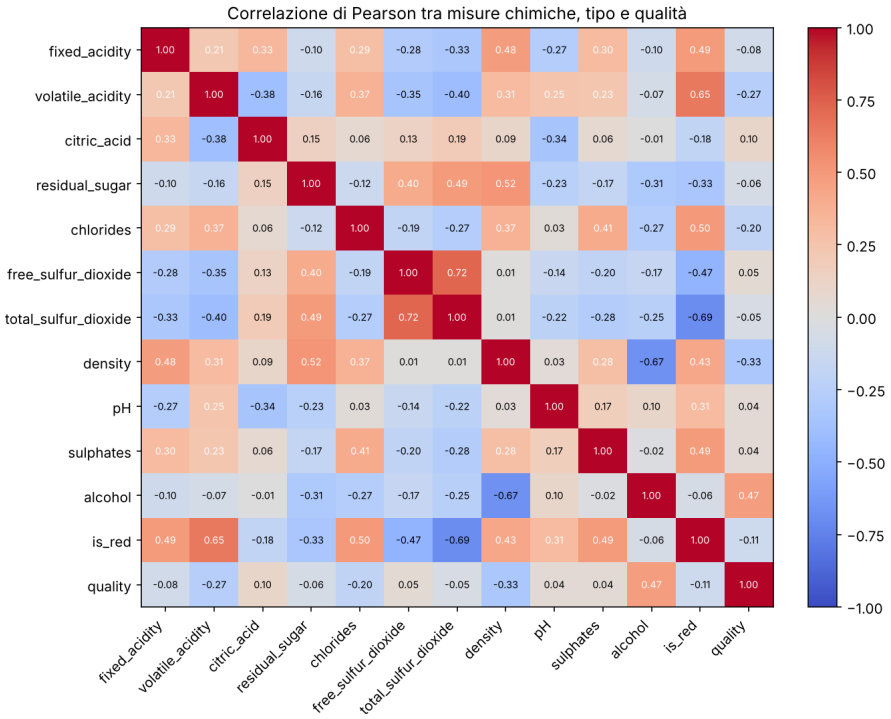


Figura 3 — Correlazione di Pearson tra le misure chimiche, il tipo di vino (`is_red`) e il punteggio di qualità, sui 5320 vini dopo la pulizia.

Normalizzazione delle feature

Le misure hanno scale molto diverse: la SO_2 totale arriva a 440 mg/L, mentre la densità varia tra 0.987 e 1.039 g/cm^3 e i cloruri tra 0.009 e 0.611 g/L. Decision Tree e Random Forest sono invarianti a trasformazioni monotone delle singole feature e non richiedono normalizzazione; la **Logistic Regression** invece sì, perché la regolarizzazione L2 e l'ottimizzatore sono sensibili alla scala. Per questo la standardizzazione (`StandardScaler`: media 0, varianza 1) è inserita nella `Pipeline` della LR:

```
# src/supervised.py — la standardizzazione fa parte del modello
elif name == "LR":
    # la regressione logistica richiede feature su scale confrontabili
    est = Pipeline([("scale", StandardScaler()),
                    ("clf", LogisticRegression(random_state=random_state, **cfg.LR_PARAMS))])

est.fit(X[tr], y[tr])      # media e varianza stimate SOLO sul training del run
```

Nota Metodologica: Poiché lo `StandardScaler` fa parte della `Pipeline`, media e deviazione standard sono stimate a ogni run solo sul training set e poi applicate al test: non c'è data leakage. Anche le feature OWL non introducono leakage: sono funzioni deterministiche delle misure con soglie fissate a priori, senza alcun parametro stimato dai dati, quindi calcolarle una volta su tutto il dataset equivale a calcolarle separatamente su training e test.

Tre configurazioni di feature

Il preprocessing produce tre matrici di feature, usate per confronti sistematici:

Configurazione	Dimensione	Descrizione	Modelli utilizzati
X_raw	(5320, 12)	11 misure chimiche + indicatore `is_red`	DT, RF, LR
X_owl	(5320, 31)	Concetti OWL binari (Background Knowledge)	BNB, BNB-EM, TAN
X_raw_owl	(5320, 43)	X_raw concatenato con X_owl	DT+OWL, RF+OWL, LR+OWL
concetti + regole	(5320, 34)	Tutti i concetti (anche ausiliari) e le 4 regole	KB (nessun apprendimento)

Il confronto tra modelli addestrati su `X_raw` e su `X_raw_owl` consente di isolare l'effetto della Background Knowledge OWL sulle prestazioni, a parità di modello. I modelli su `X_owl` misurano invece quanto la rappresentazione binaria dei concetti, da sola, conservi l'informazione utile.

```
# main.py — costruzione delle matrici di feature
concepts = build_concepts(df)                          # 34 concetti booleani
X_owl = owl_feature_matrix(concepts)                   # (5320, 31)  uint8
X_raw = prep.raw_feature_matrix(df)                    # (5320, 12)  11 misure + is_red
X_raw_owl = np.hstack([X_raw, X_owl.astype(float)])    # (5320, 43)
```

Apprendimento Supervisionato

Introduzione

L'apprendimento supervisionato è stato utilizzato per predire la classe di qualità di un vino, modellando il problema come classificazione multiclasse con tre classi ordinali:

$$\text{quality_class} \in \{ \text{bassa}, \text{media}, \text{alta} \}$$

Il problema non è banale: il giudizio di riferimento è sensoriale, quindi soggettivo e rumoroso, e le tre classi si sovrappongono ampiamente nello spazio delle misure; la classe *media*, in particolare, sta "in mezzo" alle altre due. Sono stati addestrati e confrontati tre modelli eterogenei — Decision Tree, Random Forest e Logistic Regression — sia su feature grezze sia su feature arricchite con la Background Knowledge OWL, tutti con **pesi di classe bilanciati** (`class_weight="balanced"`, `"balanced_subsample"` per la RF) per compensare lo sbilanciamento.

Librerie utilizzate

- **pandas** e **NumPy** per la gestione dei dati e la costruzione delle matrici di feature;
- **scikit-learn** per i modelli di classificazione, la `Pipeline` con `StandardScaler`, gli split stratificati, la ricerca degli iperparametri (`StratifiedShuffleSplit`, `StratifiedKFold`, `GridSearchCV`) e le metriche;
- **SciPy** per il test di Friedman, la distribuzione del range studentizzato (Nemenyi) e `logsumexp` nell'EM;
- **matplotlib** per i grafici (barplot, boxplot, heatmap, curva di apprendimento, struttura della rete).

Suddivisione e protocollo di valutazione

La valutazione è basata su **StratifiedShuffleSplit** con 10 split indipendenti (test_size = 30%): 3724 vini di training e 1596 di test per run. La stratificazione garantisce che ogni split mantenga la distribuzione originale delle tre classi. Tutti i modelli vedono **le stesse partizioni**, condizione necessaria per il test di Friedman; gli split sono generati con `random_state = 42` e il seed dei modelli nel run *r* è 42 + *r*.

```
# src/preprocessing.py
def make_splits(y, n_runs, test_size, seed):
    sss = StratifiedShuffleSplit(n_splits=n_runs, test_size=test_size, random_state=seed)
    return [(tr, te) for tr, te in sss.split(np.zeros(len(y)), y)]

# main.py — ciclo dei run (estratto)
splits = prep.make_splits(y, args.runs, cfg.TEST_SIZE, args.seed)   # 10 run, 70/30, seed 42
for run, (tr, te) in enumerate(splits):
    seed_r = args.seed + run
    for X, suffix in ((X_raw, ""), (X_raw_owl, "+OWL")):
        for m in SUPERVISED_MODELS:                                  # "DT", "RF", "LR"
            est = build_estimator(m, seed_r, tune=args.tune)
            est.fit(X[tr], y[tr])
            preds[m + suffix] = est.predict(X[te])
```

Per ogni run e ogni modello sono calcolate le metriche macro-medie sulle classi (`src/evaluation.py`): Precision, Recall, F1 e la **G-Mean**, media geometrica delle recall per classe:

$$\text{G-Mean} = \left(\text{recall}_{\text{bassa}} \cdot \text{recall}_{\text{media}} \cdot \text{recall}_{\text{alta}} \right)^{1/3}$$

La G-Mean vale 0 se anche una sola classe non viene mai riconosciuta, quindi premia i modelli che trattano bene tutte le classi, compresa la minoritaria *alta*.

> **Nota Metodologica:** La scelta di 10 run con StratifiedShuffleSplit, invece di un singolo split train/test, garantisce risultati più stabili e consente di riportare **media e deviazione standard** delle metriche. Un singolo run può dipendere dalla casualità dello split e produrre stime sovrastimate o sottostimate. Non vengono presentate le matrici di confusione dei singoli run ma tabelle aggregate (media ± std); le matrici di confusione mostrate in seguito sono la somma sui 10 run.

Ottimizzazione degli iperparametri

Gli iperparametri dell'esperimento principale sono fissati in `config.py` (`DT_PARAMS`, `RF_PARAMS`, `LR_PARAMS`). Con l'opzione `--tune` ogni modello esegue invece una **ricerca a griglia** con `StratifiedKFold` a 3 fold all'interno del solo training set di ciascun run: il test set rimane completamente separato dall'ottimizzazione.

```
# config.py — griglie per la ricerca opzionale (python main.py --tune)
TUNE_GRIDS = {
    "DT": {"max_depth": [4, 6, 8, 12], "min_samples_leaf": [1, 5, 20]},
    "RF": {"min_samples_leaf": [1, 2, 5], "max_features": ["sqrt", 0.5]},
    "LR": {"clf__C": [0.1, 1.0, 10.0]},
}

# src/supervised.py
if tune:
    cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=random_state)
    est = GridSearchCV(est, cfg.TUNE_GRIDS[name], scoring="f1_macro", cv=cv, n_jobs=1)
```

La metrica di ottimizzazione è la **F1 macro**, scelta perché bilancia precision e recall su tutte le classi, inclusa la minoritaria *alta*. Il confronto tra iperparametri fissi e ottimizzati è riportato al termine della sezione.

Decision Tree (DT)

Il Decision Tree è configurato con criterio di splitting **Gini**, profondità massima limitata e un numero minimo di esempi per foglia, per controllare l'overfitting.

Iperparametri

```
DT_PARAMS = dict(max_depth=8,                # profondità massima dell'albero
                 min_samples_leaf=5,         # almeno 5 vini per foglia
                 class_weight="balanced")    # pesi inversamente proporzionali alla frequenza
# Parametri impliciti: criterion = 'gini', random_state = 42 + r
```

Scelta di max_depth

Per verificare la scelta, la ricerca a griglia di `--tune` (CV interna a 3 fold sul solo training) è stata eseguita in ciascuno dei 10 run. La tabella riporta quante volte è stata selezionata ciascuna profondità:

max_depth	Frequenza su 10 run	min_samples_leaf scelto
6	6/10	1 (2×), 5 (3×), 20 (1×)
8	2/10	5 (1×), 20 (1×)
4	1/10	5 (1×)
12	1/10	20 (1×)

A completamento, la tabella seguente mostra la F1 macro media sui 10 run al variare di `max_depth` (con `min_samples_leaf = 5`):

max_depth	F1 macro media	F1 std	Osservazione
2	0.490	0.012	Underfitting: albero troppo semplice
4	0.517	0.018	
6	0.523	0.016	Massimo della curva
8	**0.522**	**0.015**	**Valore usato (config.py)**
10	0.515	0.015	
12	0.510	0.010	
16	0.504	0.011	Inizia l'overfitting
None (illimitato)	0.503	0.009	Albero illimitato: overfitting

La profondità 6 è preferita nella maggioranza dei run (6/10), ma la curva è piatta tra 6 e 8 (differenza di 0.002, molto inferiore alla deviazione standard): il valore 8 usato in `config.py` cade nel plateau. Alberi più profondi peggiorano progressivamente (overfitting), alberi molto bassi non hanno capacità sufficiente. Il DT ottimizzato con `--tune` ottiene F1 = 0.526, contro 0.522 con i parametri fissi.

Risultati — DT (raw) vs DT (+OWL)

```
=== Risultati: DT (Raw) ===
Precision macro : 0.518 ± 0.015
Recall macro    : 0.555 ± 0.014
F1 macro        : 0.522 ± 0.015
G-Mean          : 0.535 ± 0.017
```

```
=== Risultati: DT+OWL (Raw+OWL) ===
Precision macro : 0.519 ± 0.011
Recall macro    : 0.557 ± 0.011
F1 macro        : 0.522 ± 0.012
G-Mean          : 0.537 ± 0.013
```

F1 per classe — DT (raw), media su 10 run

Classe	F1 media	F1 std	Osservazione
bassa	0.643	0.016	Classe meglio separata (alcol basso, difetti)
media	0.428	0.033	La più difficile: classe "di confine" tra le altre due
alta	0.494	0.025	Minoritaria (19%), ma ben caratterizzata dall'alcol alto

Valutazione generale

Il Decision Tree mostra prestazioni moderate (F1 = 0.522). La classe *media* (F1 = 0.428) è la più critica: un albero divide lo spazio con soglie su singole misure e la classe intermedia, che si sovrappone a entrambe le altre, finisce frammentata in molte foglie poco pure. L'aggiunta delle feature OWL non cambia nulla (F1: 0.522 → 0.522, ΔF1 = +0.001; la versione +OWL è migliore in 5 run su 10): i concetti di soglia sono split su singole misure che l'albero sa già trovare da solo, e le congiunzioni dei concetti composti si ottengono con split successivi.

Random Forest (RF)

Il Random Forest è un ensemble di alberi decisionali, ciascuno addestrato su un campione bootstrap del training con un sottoinsieme casuale di feature candidate a ogni split. Riduce la varianza rispetto al singolo albero mediando le predizioni.

Parametri e motivazione

```
RF_PARAMS = dict(n_estimators=300,                    # 300 alberi: convergenza OOB verificata
                 min_samples_leaf=2,                  # foglie quasi pure, ma non singoletti
                 max_features="sqrt",                 # sqrt(n_feature) candidate a ogni split
                 class_weight="balanced_subsample",   # pesi ricalcolati su ogni bootstrap
                 n_jobs=-1)                           # parallelizzazione su tutti i core
```

Analisi convergenza OOB

Per verificare il numero di alberi, è stata analizzata l'accuratezza **Out-of-Bag** (OOB) al variare di `n_estimators`, sul training set del primo run (3724 vini, feature Raw). L'OOB usa come validation set interno, per ogni albero, i campioni rimasti fuori dal suo bootstrap:

n_estimators	OOB Accuracy	OOB F1 macro	Variazione rispetto alla riga precedente
10	0.558	0.541	—
25	0.585	0.570	+2.7 punti
50	0.597	0.582	+1.3 punti
100	0.610	0.596	+1.3 punti
200	0.611	0.593	+0.1 punti
300	**0.617**	**0.599**	**+0.6 punti**
500	0.621	0.604	+0.4 punti

L'accuratezza OOB cresce rapidamente fino a 100 alberi (0.610) e poi si stabilizza: da 100 a 500 alberi guadagna solo 1.1 punti. La scelta di 300 alberi si colloca nel plateau ed è un buon compromesso tra accuratezza e costo computazionale: passare a 500 costerebbe il 67% di tempo in più per 0.4 punti di accuratezza OOB.

Risultati — RF (raw) vs RF (+OWL)

```
=== Risultati: RF (Raw) ===
Precision macro : 0.618 ± 0.011
Recall macro    : 0.601 ± 0.010
F1 macro        : 0.608 ± 0.010
G-Mean          : 0.595 ± 0.010
```

```
=== Risultati: RF+OWL (Raw+OWL) ===
Precision macro : 0.608 ± 0.009
Recall macro    : 0.599 ± 0.009
F1 macro        : 0.603 ± 0.008
G-Mean          : 0.594 ± 0.009
```

F1 per classe — RF (raw), media su 10 run

Classe	F1 media	F1 std	Confronto con DT
bassa	0.694	0.012	+0.050 vs DT
media	0.595	0.016	+0.167 vs DT (il guadagno maggiore)
alta	0.535	0.013	+0.041 vs DT

Valutazione generale

Il Random Forest è il **modello migliore** dell'intera valutazione (F1 = 0.608 ± 0.010, G-Mean = 0.595) e supera il Decision Tree in tutte le classi, soprattutto sulla classe *media* (+0.167): l'averaging di 300 alberi rende la frontiera tra classi sovrapposte più stabile, come mostra anche la deviazione standard più bassa (F1 std 0.010 contro 0.015). L'aggiunta della BK OWL peggiora leggermente la F1 (ΔF1 = −0.005; la versione +OWL è migliore in 1 run su 10). Un'ipotesi plausibile è la diluizione delle feature: con `max_features="sqrt"` ogni split sceglie tra circa 6 feature candidate su 43 invece che tra 3 su 12, e la maggior parte delle candidate diventano concetti binari, versioni più grossolane delle stesse misure continue che la foresta sa già sfruttare.

Logistic Regression (LR)

La Logistic Regression multinomiale apprende confini di decisione lineari nello spazio delle feature standardizzate. Utilizza il solver `lbfgs` (default di scikit-learn), adatto a problemi multiclasse con dataset di dimensione media.

Iperparametri

```
LR_PARAMS = dict(C=1.0,                    # inverso della forza di regolarizzazione L2
                 max_iter=3000,            # convergenza garantita
                 class_weight="balanced")
# Pipeline: StandardScaler -> LogisticRegression (solver 'lbfgs', random_state = 42 + r)
```

Sensitivity analysis — parametro C

La tabella seguente mostra l'effetto di C sulla F1 macro (media ± std sui 10 run), sia su Raw sia su Raw+OWL. È un'analisi descrittiva: il valore usato resta quello di `config.py`.

C	F1 macro (Raw)	F1 macro (Raw+OWL)	Effetto
0.01	0.548 ± 0.010	0.539 ± 0.009	Regolarizzazione forte: lieve underfitting
0.1	0.551 ± 0.011	0.547 ± 0.010	Praticamente al massimo
1.0	**0.552 ± 0.012**	**0.545 ± 0.009**	**Valore usato (config.py)**
10	0.551 ± 0.012	0.546 ± 0.009	Nessuna differenza apprezzabile
100	0.551 ± 0.011	0.546 ± 0.009	Nessuna differenza apprezzabile

La LR è praticamente insensibile a C per C ≥ 0.1 (variazioni ≤ 0.001): con 3724 esempi di training e solo 12 (o 43) feature il modello non è in regime di overfitting, quindi la regolarizzazione conta poco. Anche una regolarizzazione forte (C = 0.01) costa appena 0.004 di F1 su Raw.

Risultati — LR (raw) vs LR (+OWL)

```
=== Risultati: LR (Raw) ===
Precision macro : 0.548 ± 0.011
Recall macro    : 0.594 ± 0.010
F1 macro        : 0.552 ± 0.012
G-Mean          : 0.567 ± 0.012
```

```
=== Risultati: LR+OWL (Raw+OWL) ===
Precision macro : 0.543 ± 0.009
Recall macro    : 0.590 ± 0.009
F1 macro        : 0.545 ± 0.009
G-Mean          : 0.560 ± 0.010
```

F1 per classe — LR (+OWL), media su 10 run

Classe	F1 media	F1 std	Confronto con LR (raw)
bassa	0.671	0.011	−0.002 vs LR (raw)
media	0.428	0.013	−0.012 vs LR (raw)
alta	0.536	0.011	−0.005 vs LR (raw)

Valutazione generale

La Logistic Regression su feature grezze (F1 = 0.552) è il secondo modello supervisionato. La sua recall (0.594) è nettamente superiore alla precision (0.548): i pesi bilanciati spostano le predizioni verso la classe minoritaria *alta*, che infatti ottiene la F1 più alta tra tutti i modelli supervisionati (0.541, contro 0.535 della RF), a scapito della classe *media* (0.441).

Contrariamente all'ipotesi iniziale (i concetti potrebbero introdurre non-linearità utili a un modello lineare), la BK OWL **non aiuta** la LR: ΔF1 = −0.006, con la versione +OWL migliore in 2 run su 10, e il calo si concentra sulla classe *media*. La misura più informativa, il grado alcolico, ha un legame quasi monotono con la qualità che la LR cattura già linearmente; i 31 concetti binari aggiungono gradini fortemente correlati alle stesse misure, aumentando la varianza più che l'informazione.

Iperparametri fissi e ottimizzati (--tune)

La tabella confronta la F1 macro dei modelli supervisionati con gli iperparametri fissi di `config.py` e con quelli selezionati dalla ricerca a griglia (`python main.py --tune`, CV interna a 3 fold in ogni run):

Modello	F1 (iperparametri fissi)	F1 (--tune)	Differenza
DT	0.522 ± 0.015	0.526 ± 0.017	+0.004
RF	0.608 ± 0.010	0.605 ± 0.009	−0.003
LR	0.552 ± 0.012	0.552 ± 0.011	+0.000
DT+OWL	0.522 ± 0.012	0.524 ± 0.013	+0.002
RF+OWL	0.603 ± 0.008	0.600 ± 0.008	−0.003
LR+OWL	0.545 ± 0.009	0.545 ± 0.009	+0.000

Le differenze sono tutte entro ±0.004, cioè ben dentro la deviazione standard tra i run: gli iperparametri di `config.py` sono già vicini all'ottimo, e le conclusioni del confronto non dipendono dall'ottimizzazione (con `--tune` il test di Friedman resta significativo e RF resta il modello migliore).

Conclusioni — Apprendimento Supervisionato

Il confronto tra i tre modelli evidenzia differenze sostanziali, mentre la Background Knowledge OWL ha un effetto trascurabile:

- **RF** è il modello migliore (F1 = 0.608 ± 0.010, G-Mean = 0.595 ± 0.010) e il più stabile: l'ensemble gestisce meglio la sovrapposizione tra le classi.
- **LR** è secondo (F1 = 0.552) e, grazie ai pesi bilanciati, è il migliore sulla classe minoritaria *alta*.
- **DT** è il più debole (F1 = 0.522) ed è il più penalizzato sulla classe *media*.
- La **BK OWL** non migliora nessun modello: ΔF1 = +0.001 (DT), −0.005 (RF), −0.006 (LR). I concetti sono soglie sulle stesse misure che i modelli vedono già: non aggiungono informazione nuova, ma solo

una sua discretizzazione.

- La classe **media** è la più difficile per tutti i modelli (F1 tra 0.43 e 0.60): è la classe di confine di un problema ordinale.

Modelli su Feature OWL: BNB e BNB-EM

Introduzione

I modelli **Bernoulli Naive Bayes** (BNB) e **BNB-EM** operano direttamente sul vettore binario dei 31 concetti OWL, trattando ogni concetto come una variabile di Bernoulli condizionatamente indipendente dalle altre data la classe. Sono implementati da zero con numpy (`src/bernoulli_nb.py`, `src/bnb_em.py`): il BNB è verificato contro `sklearn.naive_bayes.BernoulliNB`, con parametri e predizioni identici (`test_bnb_matches_sklearn`).

La scelta dei modelli di Bernoulli è coerente con la natura binaria delle feature OWL: a differenza della Logistic Regression, che tratta le feature come variabili numeriche, il BNB assume esplicitamente che ogni concetto sia il risultato di un lancio di moneta con parametro θ dipendente dalla classe.

Il protocollo è **semi-supervisionato**: in ogni run solo il 20% del training mantiene l'etichetta (745 vini, campionati in modo stratificato da `labeled_mask`). Il BNB usa solo questi; il BNB-EM usa anche gli altri 2979 vini del training, di cui ignora l'etichetta.

Bernoulli Naive Bayes (BNB)

Formulazione matematica

Il modello BNB assume indipendenza condizionale delle feature data la classe:

```
P(c | x)  ∝  P(c) · Π_j  θ_cj^x_j · (1 − θ_cj)^(1 − x_j)

Con smoothing di Laplace α:
  θ_cj = (N_cj + α) / (N_c + 2α)            P(c) = (N_c + α) / (N + K·α)

dove:
  N_cj = numero di vini della classe c con il concetto j attivo (x_j = 1)
  N_c  = numero di vini della classe c,   N = totale,   K = 3 classi
  α    = parametro di smoothing (config.BNB_ALPHA = 1.0)

Classificazione:  ŷ = argmax_c [ log P(c) + Σ_j log P(x_j | c) ]
```

Sensitivity analysis — parametro α

Lo smoothing di Laplace α previene probabilità nulle per concetti mai osservati in una classe. La tabella mostra l'impatto di α sulla F1 macro (10 run, 20% di etichette):

α	F1 macro media	F1 std	Osservazione
0.01	0.504	0.009	Smoothing quasi nullo
0.1	0.505	0.010	
0.5	0.504	0.009	
1.0	**0.505**	**0.010**	**Laplace standard (valore usato)**
2	0.506	0.010	
5	0.510	0.009	Lieve miglioramento
10	0.517	0.007	Migliore sul dataset corrente

Il modello è insensibile ad α nell'intervallo [0.01, 2] (F1 tra 0.504 e 0.506), mentre uno smoothing più forte migliora leggermente (+0.012 con α = 10). Un'interpretazione plausibile: lo smoothing avvicina i θ a 0.5 e smorza i rapporti di verosimiglianza; poiché le feature sono correlate (concetti mutuamente esclusivi di una famiglia, composti che ripetono i loro componenti), il Naive Bayes conta più volte la stessa evidenza e diventa troppo sicuro, e uno smoothing più forte compensa in parte. Il valore α = 1 (Laplace standard, `BNB_ALPHA`) è stato mantenuto: sceglierlo in base a questa tabella significherebbe selezionarlo sul test.

Implementazione

```
# src/bernoulli_nb.py — estratto semplificato
def _m_step(self, X, R, w=None):
    """Massimizzazione con conteggi soft: R (n, K) responsabilità, w (n,) pesi degli esempi."""
    a, K = self.alpha, self.n_classes
    Rw = R if w is None else R * w[:, None]
    Nk = Rw.sum(axis=0)                                  # massa di ogni classe
    Njk = Rw.T @ X.astype(float)                         # massa con x_j = 1 per classe
    self.class_log_prior_ = np.log((Nk + a) / (Nk.sum() + K * a))
    theta = (Njk + a) / (Nk[:, None] + 2.0 * a)
    self.log_theta_, self.log_1m_theta_ = np.log(theta), np.log1p(-theta)

def fit(self, X, y):                                     # etichette -> responsabilità one-hot
    R = np.zeros((len(y), self.n_classes)); R[np.arange(len(y)), y] = 1.0
    self._m_step(np.asarray(X), R)
    return self

def log_joint(self, X):                            # log P(c) + log P(x | c), matrice (n, K)
    X = np.asarray(X, dtype=float)
    return X @ self.log_theta_.T + (1.0 - X) @ self.log_1m_theta_.T + self.class_log_prior_
```

```
=== Risultati: BNB (OWL, 20% etichettati, α = 1.0) ===
Precision macro : 0.502 ± 0.009
Recall macro    : 0.532 ± 0.009
F1 macro        : 0.505 ± 0.010
G-Mean          : 0.512 ± 0.010
```

Con il 100% delle etichette lo stesso modello raggiunge F1 = 0.518 ± 0.007: ridurre le etichette a un quinto costa solo 0.013 di F1.

BNB-EM — Apprendimento Semi-Supervisionato

Motivazione

Nel dominio enologico l'annotazione è costosa: ogni etichetta richiede la degustazione da parte di almeno tre esperti, mentre le misure chimiche si ottengono con strumenti di laboratorio. È quindi realistico disporre di molti vini analizzati e di pochi vini degustati. Il BNB-EM affronta questo scenario tramite l'algoritmo **Expectation-Maximization**, che usa anche i vini non etichettati per migliorare la stima dei parametri.

Algoritmo EM

```
Algoritmo BNB-EM (src/bnb_em.py; Nigam, McCallum, Thrun, Mitchell, 2000)

INPUT : X_l, y_l (etichettati: 20% del training), X_u (non etichettati), λ ∈ [0, 1]
OUTPUT: parametri θ_cj e P(c)

1. INIZIALIZZAZIONE: M-step sui soli esempi etichettati (= BNB supervisionato)

2. RIPETI fino a convergenza (al più EM_MAX_ITER = 100 iterazioni):
   E-STEP:  per ogni x ∈ U:   γ_c(x) = P(c | x, θ)          [Σ_c γ_c(x) = 1]
   M-STEP:  N_c  = Σ_L 1[y = c]        + λ · Σ_U γ_c(x)
            N_cj = Σ_L 1[y = c] · x_j  + λ · Σ_U γ_c(x) · x_j
            θ_cj = (N_cj + α) / (N_c + 2α)

   log L = Σ_L log P(x, y)  +  λ · Σ_U log Σ_c P(x, c)
   CONVERGENZA: |log L − log L_prec| ≤ EM_TOL · |log L|      (EM_TOL = 1e-5)
```

Il parametro λ pesa il contributo dei non etichettati (λ = 1: peso pieno, valore usato nell'esperimento principale). Il test `test_em_objective_is_monotone_and_helps_when_assumptions_hold` verifica che l'obiettivo (con il prior di Laplace) non decresca mai e che l'EM migliori il BNB quando le ipotesi del modello valgono davvero (dati sintetici generati da un Naive Bayes).

Convergenza dell'algoritmo

Nel primo run (λ = 1) l'algoritmo converge in **15 iterazioni**; sui 10 run il numero medio di iterazioni è 18.2 (da 14 a 28).

Iterazione	Log-verosimiglianza (log L)	Δ rispetto all'iterazione precedente
1	−52866.97	—
2	−51081.62	+1785.35
3	−50601.39	+480.23
4	−50230.81	+370.58
5	−49876.32	+354.49
6	−49631.46	+244.86
8	−49423.12	+59.88
10	−49397.12	+7.13
12	−49391.82	+1.89
15 (convergenza)	−49389.55	+0.44

La log-verosimiglianza cresce in modo monotono; il guadagno più grande avviene alla seconda iterazione (+1785) e dopo una decina di iterazioni le variazioni diventano trascurabili. Il criterio relativo 1e-5 ferma l'algoritmo all'iterazione 15, quando la variazione relativa scende a 9.0e-06.

Sensitivity — percentuale di dati etichettati

La **curva di apprendimento** (`run_curve` in `main.py`) ripete il confronto sugli stessi 10 run con l'1, 2, 5, 10, 20, 50 e 100% di etichette e con due pesi dei non etichettati (λ = 1 e λ = 0.1). F1 macro, media ± std:

% etichettati	BNB	BNB-EM (λ = 1)	BNB-EM (λ = 0.1)	EM (λ = 1) vs BNB
1%	0.464 ± 0.027	0.423 ± 0.051	0.426 ± 0.040	−0.041
2%	0.492 ± 0.026	0.415 ± 0.032	0.420 ± 0.014	−0.077
5%	0.500 ± 0.015	0.411 ± 0.015	0.432 ± 0.012	−0.089
10%	0.512 ± 0.010	0.414 ± 0.010	0.470 ± 0.029	−0.098
20%	**0.505 ± 0.010**	**0.424 ± 0.008**	**0.490 ± 0.007**	**−0.081 (configurazione testata)**
50%	0.513 ± 0.008	0.459 ± 0.010	0.509 ± 0.009	−0.054
100%	0.518 ± 0.007	0.518 ± 0.007	0.518 ± 0.007	+0.000

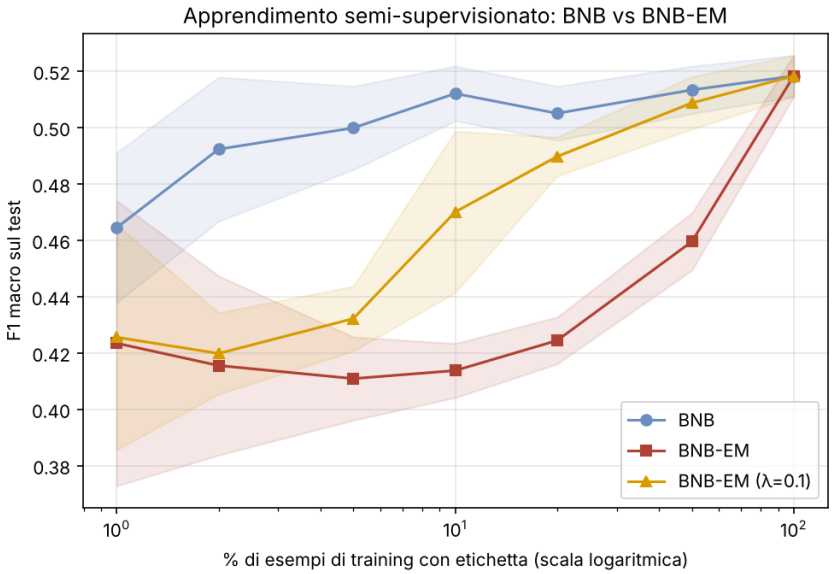


Figura 4 — Curva di apprendimento semi-supervisionato: F1 macro sul test al variare della percentuale di esempi di training etichettati (scala logaritmica). Le bande indicano ± una deviazione standard sui 10 run.

```
=== Risultati: BNB-EM (OWL, 20% etichettati + 80% non etichettati, λ = 1) ===
Precision macro : 0.431 ± 0.008
Recall macro    : 0.468 ± 0.009
F1 macro        : 0.424 ± 0.008
G-Mean          : 0.446 ± 0.009
```

Perché l'EM peggiora: l'analisi dei cluster

Per capire il degrado sono state analizzate, nel primo run, le predizioni del BNB-EM sul test. Tra i vini che il BNB-EM classifica *bassa*, l'**88.3% sono rossi**, mentre tra quelli classificati *media* e *alta* i rossi sono appena lo 0.3% e l'1.0%; nella realtà i rossi sono il 28.9%, il 24.0% e il 17.8% delle tre classi. L'EM massimizza la verosimiglianza dei dati, non l'accuratezza: i "cluster" che trova seguono la struttura più evidente dei concetti — la separazione tra rossi e bianchi, che differiscono nettamente per cloruri, solfati, acidità volatile e solforosa — invece della qualità, e di fatto la classe *bassa* diventa "vino rosso". Il prior stimato si deforma di conseguenza: P(alta) passa dal 19.0% del BNB al 32.8% del BNB-EM.

È il fenomeno descritto da Cozman e Cohen (2002): quando le ipotesi del modello generativo sono violate — e qui l'indipendenza condizionale lo è certamente — i dati non etichettati possono peggiorare il classificatore. Con λ = 0.1 l'effetto si attenua (i rossi tra i vini predetti *bassa* scendono al 55.5%) e la perdita di F1 al 20% di etichette si riduce da −0.081 a −0.015.

Valutazione generale

Il BNB supervisionato (F1 = 0.505) supera il BNB-EM (F1 = 0.424) per ogni quota di etichette dall'1% al 50%; la risposta a **RQ2** è quindi negativa per questo dominio. Il risultato positivo è la **compattezza della rappresentazione ontologica**: con appena l'1% delle etichette (37 vini) il BNB raggiunge F1 = 0.464, e con tutte le etichette 0.518. Uno sviluppo naturale è modellare esplicitamente il tipo di vino, stimando P(x | classe, tipo), così che l'EM non debba "riscoprire" la separazione tra rossi e bianchi.

Knowledge Base Prolog

Introduzione

Nel sistema ENOKON la **Knowledge Base Prolog** (`kb/wine_kb.pl`) svolge il ruolo di livello simbolico complementare all'ontologia OWL. Mentre l'ontologia definisce i concetti tramite restrizioni sulle data property, la KB esprime gli stessi concetti come **clausole di Horn** e aggiunge le **regole di classificazione** di un sistema esperto, con la spiegazione della decisione.

La KB non è scritta a mano: è generata da `kb/build_kb.py` a partire da `src/knowledge.py`, la stessa fonte dell'ontologia e delle feature. Non è concepita come semplice archivio di dati, ma come modulo di ragionamento esplicito integrato nel sistema: le sue regole formalizzano conoscenza enologica in forma dichiarativa, consentendo inferenza, classificazione e spiegazione. Soprattutto, **non apprende e non vede mai le etichette**: le regole sono ipotesi a priori dell'esperto.

Struttura della Knowledge Base

Il file `wine_kb.pl` è strutturato in quattro componenti:

- **Fatti** (uno per misura, per ogni vino): `wine(Id)`, `tipo(Id, rosso|bianco)` e 11 predicati di misura come `grado_alcolico(Id, 12.5)` o `ph(Id, 3.2)`, dichiarati `:- dynamic`.
- **28 regole per i concetti di soglia e di tipo**, ad esempio `alcol_alto(V) :- grado_alcolico(V, X), X >= 12.0.`
- **11 clausole per i 6 concetti composti**: le congiunzioni diventano corpi di clausola e le disgiunzioni più clausole con la stessa testa, secondo la forma normale disgiuntiva calcolata da `knowledge.to_dnf`.
- **4 regole di classificazione** e il predicato di spiegazione `classifica/3` (più `classifica/2`).

Fatti e regole — Esempi

Concetti di soglia e di tipo

```
% wine_kb.pl — concetti di soglia e di tipo (estratto)
vino_rosso(V)               :- tipo(V, rosso).
acidita_volatile_bassa(V)   :- acidita_volatile(V, X), X < 0.3.
acidita_volatile_elevata(V) :- acidita_volatile(V, X), X >= 0.6.
abboccato(V)                :- zuccheri_residui(V, X), X > 4.0, X =< 12.0.
so2_libera_bassa(V)         :- so2_libera(V, X), X < 15.0.
so2_totale_oltre_150(V)     :- so2_totale(V, X), X > 150.0.
ph_alto(V)                  :- ph(V, X), X >= 3.5.
solfati_alti(V)             :- solfati(V, X), X >= 0.75.
alcol_alto(V)               :- grado_alcolico(V, X), X >= 12.0.
```

Concetti composti — coerenza con OWL

```
% SO2TotaleEccessiva := (VinoRosso ∧ SO2TotaleOltre150) ∨ (VinoBianco ∧ SO2TotaleOltre200)
so2_totale_eccessiva(V) :- vino_rosso(V), so2_totale_oltre_150(V).
so2_totale_eccessiva(V) :- vino_bianco(V), so2_totale_oltre_200(V).
% RischioInstabilita := PHAlto ∧ SO2LiberaBassa
rischio_instabilita(V)  :- ph_alto(V), so2_libera_bassa(V).
% VinoDifettoso := AciditaVolatileElevata ∨ SO2TotaleEccessiva ∨ RischioInstabilita
vino_difettoso(V) :- acidita_volatile_elevata(V).
vino_difettoso(V) :- so2_totale_eccessiva(V).
vino_difettoso(V) :- rischio_instabilita(V).
% VinoMaturoEPulito := AlcolAlto ∧ AciditaVolatileBassa
vino_maturo_e_pulito(V) :- alcol_alto(V), acidita_volatile_bassa(V).
```

Regole di classificazione e spiegazione

```
% Le regole sono provate in ordine di priorità (bassa > alta > media):
% la prima che scatta decide.
regola_bassa(V, difetto_enologico)           :- vino_difettoso(V).
regola_bassa(V, poco_alcol_pochi_solfati)    :- alcol_basso(V), solfati_bassi(V).
regola_alta(V, maturo_e_pulito)              :- vino_maturo_e_pulito(V).
regola_alta(V, equilibrato_con_solfati_alti) :- vino_equilibrato(V), solfati_alti(V).

% classifica(+Vino, ?Classe, ?Motivo): classe di qualità e regola che l'ha decisa
classifica(V, C, Motivo) :-
    (   regola_bassa(V, M) -> C = bassa, Motivo = M
    ;   regola_alta(V, M)  -> C = alta,  Motivo = M
    ;   C = media, Motivo = nessuna_regola
    ).
classifica(V, C) :- classifica(V, C, _).
```

Regola	Classe	Condizione	Motivazione
difetto_enologico	bassa	VinoDifettoso	Un difetto analitico (spunto acetico, eccesso di SO_2, rischio di instabilità) penalizza il vino
poco_alcol_pochi_solfati	bassa	AlcolBasso ∧ SolfatiBassi	Poca struttura e poca protezione antiossidante
maturo_e_pulito	alta	VinoMaturoEPulito	Uve mature (alcol alto) e fermentazione pulita
equilibrato_con_solfati_alti	alta	VinoEquilibrato ∧ SolfatiAlti	Profilo equilibrato con buona protezione
(default)	media	—	Nessuna regola applicabile

Spiegabilità — esempio di interrogazione

```
?- [kb/wine_kb].
?- assertz(wine(v1)), assertz(tipo(v1, bianco)),
   assertz(grado_alcolico(v1, 12.5)), assertz(acidita_volatile(v1, 0.22)),
   assertz(acidita_fissa(v1, 6.8)), assertz(acido_citrico(v1, 0.3)),
   assertz(zuccheri_residui(v1, 2.0)), assertz(cloruri(v1, 0.04)),
   assertz(so2_libera(v1, 30.0)), assertz(so2_totale(v1, 110.0)),
   assertz(densita(v1, 0.991)), assertz(ph(v1, 3.2)), assertz(solfati(v1, 0.5)).
?- classifica(v1, Classe, Motivo).
Classe = alta, Motivo = maturo_e_pulito.
```

Il vino **v1** soddisfa **AlcolAlto** (12.5 % vol) e **AciditaVolatileBassa** (0.22 g/L), quindi è un **VinoMaturoEPulito**. Nessuna regola di priorità più alta scatta: l'acidità volatile è sotto 0.6 g/L, la SO_2 totale (110 mg/L) è sotto il limite dei bianchi e il pH (3.2) non è alto, quindi il vino non è difettoso, e l'alcol non è basso. La KB risponde *alta* e indica la regola che ha deciso: è la **spiegazione** che i modelli statistici non forniscono. Lo stesso esempio, valutato con il motore Python, attiva esattamente i concetti **VinoBianco**, **AciditaVolatileBassa**, **Secco**, **CloruriBassi**, **SO2LiberaAdeguata**, **DensitaBassa**, **PHMedio**, **AlcolAlto** e **VinoMaturoEPulito**.

Integrazione Python–Prolog

L'integrazione è gestita da **src/prolog_kb.py**. Se l'eseguibile **SWI-Prolog** (**swipl**) è nel PATH, i fatti di tutti i vini vengono scritti in un file temporaneo insieme alla KB e a un predicato **main**, e la KB viene interrogata davvero in Prolog con una sola invocazione; altrimenti si usa il motore di regole Python equivalente (**knowledge.apply_rules**).

```
# src/prolog_kb.py — estratto semplificato
_MAIN = """
main :- forall(wine(V), ( classifica(V, C), format("~w ~w~n", [V, C]) )).
"""

def facts_text(df):                       # fatti di tutti i vini, raggruppati per predicato
    lines = [f"wine(w{i})." for i in range(len(df))]
    lines += [f"tipo(w{i}, {t})." for i, t in enumerate(df["tipo"])]
    for p in PROPERTIES:                  # grado_alcolico(w0, 9.400000). ...
        vals = df[p.column].to_numpy(dtype=float)
        lines += [f"{p.pl}(w{i}, {vals[i]:.6f})." for i in range(len(df))]
    return "\n".join(lines) + "\n"

def _run_swipl(self, df):
    exe = shutil.which("swipl")
    if exe is None:                       # graceful degradation -> motore Python
        self.warning = "SWI-Prolog (swipl) non trovato nel PATH"
        return None
    program = self.kb_path.read_text(encoding="utf-8") + "\n" + facts_text(df) + _MAIN
    # ... scrittura in un file temporaneo ...
    proc = subprocess.run([exe, "-q", "-g", "main", "-t", "halt", str(f)],
                          capture_output=True, text=True, timeout=900)
    # ogni riga "w<i> <classe>" diventa la predizione del vino i
```

Nota Metodologica: Il sistema include **graceful degradation**: se SWI-Prolog non è installato o l'esecuzione fallisce, il modulo ripiega sul motore Python e il motivo viene riportato in **results/summary.md** (con **--kb-engine swipl** si forza invece Prolog, con errore se non disponibile). Quando gira SWI-Prolog, il risultato è confrontato con quello del motore Python e la percentuale di accordo, attesa del 100%, è un controllo di coerenza. Ogni vino ha un identificatore proprio (**w0** … **w5319**), quindi non c'è contaminazione tra vini diversi.

L'equivalenza tra Prolog e motore Python è garantita anche dal test `test_prolog_matches_python`, che valuta le clausole generate con un interprete Prolog minimale indipendente su 250 vini casuali e sui casi limite, confrontando sia i 34 concetti sia la classe finale. Nell'esecuzione i cui risultati sono riportati in questo documento la KB è stata valutata dal motore Python equivalente, perché SWI-Prolog non era installato.

Copertura e precisione delle regole

Poiché la KB non apprende, ha senso valutare ogni regola sull'intero dataset: quanti vini decide (come prima regola che scatta) e con quale precisione (`results/kb_rules.csv`):

Regola	Classe	Vini decisi	Copertura	Precisione	Frequenza a priori della classe
difetto_enologico	bassa	877	16.5%	60.5%	37.4%
poco_alcol_pochi_solfati	bassa	746	14.0%	58.6%	37.4%
maturo_e_pulito	alta	339	6.4%	45.7%	19.0%
equilibrato_con_solfati_alti	alta	35	0.7%	34.3%	19.0%
(default)	media	3323	62.5%	46.8%	43.7%

Le regole per la classe *bassa* sono le più precise (60.5% e 58.6%, contro una frequenza a priori del 37.4%) e coprono insieme il 30.5% dei vini. Le regole per la classe *alta* sono meno precise (45.7% e 34.3%), ma comunque ben sopra la frequenza a priori (19.0%), e soprattutto coprono pochissimi vini (7.1% in totale): di conseguenza il 62.5% dei vini finisce nella classe di default *media*.

Complessità del ragionamento

Il ragionamento avviene per **backward chaining** (risoluzione SLD, standard in Prolog). Sia J il numero di vini e R il numero di regole di classificazione:

- ogni concetto di soglia richiede la lettura di un fatto e un confronto aritmetico: grazie all'indicizzazione sul primo argomento (l'identificatore del vino) il costo è costante; un concetto composto in forma normale disgiuntiva con d disgiunti prova al più d clausole;
- `classifica/3` prova al più le R = 4 regole in ordine di priorità e il costrutto *if-then-else* (`->`) si impegna sulla prima che ha successo, senza backtracking sulle alternative;
- la classificazione di J vini ha quindi complessità O(J × R), lineare nel numero di vini: per i 5320 vini del dataset è immediata.

Rete Bayesiana

Introduzione

Oltre ai modelli discriminativi (DT, RF, LR) e al Naive Bayes, nel progetto ENOKON è stato implementato un modello probabilistico basato su inferenza bayesiana. La rete bayesiana consente di modellare esplicitamente le dipendenze tra le variabili del dominio e di calcolare la probabilità condizionata della classe di qualità date le evidenze (i concetti OWL):

$$P(\text{Qualità} \mid \text{concetto}_1, \text{concetto}_2, \ldots, \text{concetto}_{31})$$

A differenza di un classificatore deterministico, la rete restituisce una distribuzione di probabilità sulle classi, fornendo un livello superiore di interpretabilità.

Struttura della rete — Tree-Augmented Naive Bayes

La struttura è un **Tree-Augmented Naive Bayes** (TAN, Friedman, Geiger e Goldszmidt, 1997), implementato da zero con numpy in `src/bayesian_network.py`. Come nel Naive Bayes il nodo *Qualità* è genitore di tutti i 31 concetti, ma in più i concetti formano un **albero**: ogni concetto ha al più un altro concetto come genitore. La scelta della struttura TAN è motivata da due ragioni:

- **Coerenza con il dominio**: l'indipendenza condizionale del Naive Bayes è certamente violata dai concetti mutuamente esclusivi di una stessa famiglia (`AlcolBasso`, `AlcolMedio`, `AlcolAlto`) e dai concetti composti, che sono funzioni dei loro componenti; il TAN può rappresentare queste dipendenze.
- **Complessità computazionale**: l'apprendimento della struttura TAN è polinomiale (informazione mutua su tutte le coppie e albero di copertura massimo) e l'inferenza esatta resta lineare nel numero di nodi, mentre l'apprendimento di una struttura generica è un problema NP-difficile.

Apprendimento della struttura e dei parametri

- **Informazione mutua condizionata** per ogni coppia di concetti, stimata con conteggi smussati: $I(X_i; X_j \mid C) = \Sigma_c P(c) \Sigma_{a,b} P(a,b \mid c) \cdot \log [P(a,b \mid c) / (P(a \mid c) P(b \mid c))]$;
- **albero di copertura di peso massimo** (algoritmo di Prim) sui pesi $I(X_i; X_j \mid C)$, orientato a partire dalla radice, scelta come il concetto con informazione mutua totale più alta;
- **tabelle di probabilità condizionata** $P(X_j \mid C, X_{parent(j)})$ con smoothing di Laplace ($\alpha = 1$);
- **inferenza esatta** grazie alla fattorizzazione: $P(c \mid x) \propto P(c) \cdot \prod_j P(x_j \mid c, x_{parent(j)})$.

```
# src/bayesian_network.py — estratto semplificato
def fit(self, X, y):
    self.cmi_ = self._conditional_mutual_information(X, y)      # (31, 31)
    root = int(np.argmax(self.cmi_.sum(axis=1)))                # concetto più "centrale"
    self.parent_ = self._maximum_spanning_tree(self.cmi_, root) # Prim: -1 = radice
    counts = np.bincount(y, minlength=K).astype(float)
    self.class_log_prior_ = np.log((counts + a) / (n + K * a))
    # CPT: theta_[j][c, v] = P(X_j = 1 | C = c, X_parent(j) = v)  (la radice ignora v)
    for j in range(d):
        th, p = np.zeros((K, 2)), self.parent_[j]
        for c in range(K):
            in_c = y == c
            if p < 0:
                th[c, :] = (X[in_c, j].sum() + a) / (in_c.sum() + 2.0 * a)
            else:
                for v in (0, 1):
                    m = in_c & (X[:, p] == v)
                    th[c, v] = (X[m, j].sum() + a) / (m.sum() + 2.0 * a)
        self.theta_.append(th)

def log_joint(self, X):                  # log P(c) + Σ_j log P(x_j | c, x_parent(j))
    lj = np.tile(self.class_log_prior_, (n, 1))
    for j in range(d):
        p = self.parent_[j]
        pv = X[:, p].astype(int) if p >= 0 else np.zeros(n, dtype=int)
        t = self.theta_[j][:, pv].T                              # (n, K)
        xj = X[:, j][:, None].astype(float)
        lj += xj * np.log(t) + (1.0 - xj) * np.log1p(-t)
    return lj
```

Nodi della rete

Nodo	Tipo	Valori	Ruolo
Qualità	Categoriale	bassa, media, alta	Nodo target (da inferire), genitore di tutti i concetti
VinoRosso	Booleano	{0, 1}	Radice dell'albero dei concetti (run 1)
AlcolBasso, AlcolMedio, AlcolAlto	Booleano	{0, 1}	Figli di Qualità e di un concetto genitore
VinoDifettoso, VinoFresco, ...	Booleano	{0, 1}	Concetti composti, collegati ai loro componenti
...	Booleano	{0, 1}	31 concetti OWL in totale (30 archi tra concetti)

Struttura appresa

Nel primo run l'albero ha radice `VinoRosso` e 30 archi tra concetti, che ricadono in quattro gruppi:

Tipo di dipendenza	Archi	Esempi (genitore → figlio, I condizionata)
Concetti della stessa famiglia (mutuamente esclusivi)	10	PHBasso → PHMedio (0.505); Secco → Abboccato (0.397); AlcolMedio → AlcolBasso (0.347)
Concetto composto e suoi componenti	8	VinoFresco → PHBasso (0.213); AciditaVolatileElevata → VinoDifettoso (0.194); AciditaVolatileBassa → VinoFresco (0.186)
Tipo di vino → altre misure	7	VinoRosso → CloruriBassi (0.215); VinoRosso → AciditaVolatileBassa (0.136); VinoRosso → SolfatiBassi (0.108)
Relazioni chimiche tra famiglie diverse	5	CloruriBassi → DensitaBassa (0.097); Amabile → DensitaAlta (0.082); AciditaVolatileElevata → AcidoCitricoBasso (0.069)

La struttura appresa **ritrova le dipendenze introdotte dalla Background Knowledge per costruzione** — l'esclusività dei concetti di una famiglia e il legame tra un composto e i suoi componenti — e in più relazioni chimiche reali: gli zuccheri alzano la densità (`Amabile → DensitaAlta`), i vini poveri di cloruri sono spesso poco densi (`CloruriBassi → DensitaBassa`), e il tipo rosso condiziona cloruri, solfati, acidità volatile e solforosa. Sono esattamente le dipendenze che il Naive Bayes ignora.

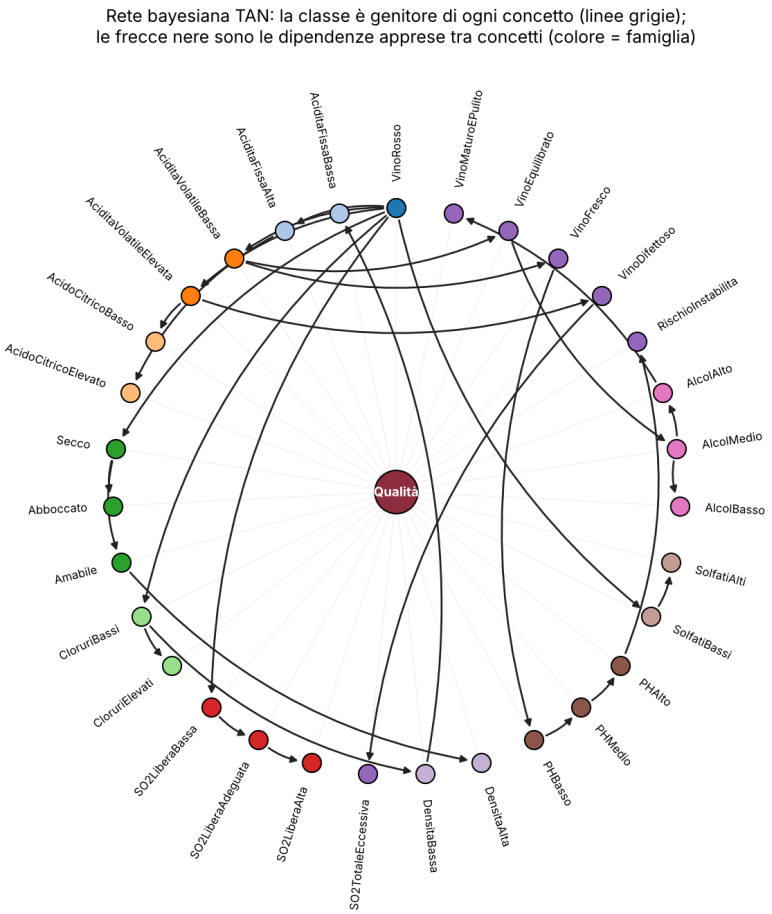


Figura 5 — Struttura della rete TAN appresa nel primo run: la classe Qualità (al centro) è genitore di ogni concetto (linee grigie); le frecce nere sono le dipendenze apprese tra concetti (colore = famiglia).

Inferenza probabilistica

Per ogni vino di test, i 31 concetti sono forniti come evidenza e viene calcolata la distribuzione a posteriori della classe. Esempio di output per un vino bianco del test del primo run, con punteggio sensoriale 7 (classe *alta*), confrontato con un Naive Bayes addestrato sulle stesse etichette:

```
Evidenza (concetti attivi; tutti gli altri 24 concetti = 0):
  Abboccato, CloruriBassi, SO2LiberaAdeguata, DensitaBassa,
  PHBasso, SolfatiBassi, AlcolAlto

                          TAN       Naive Bayes
P(bassa | evidenza) = 0.001       0.004
P(media | evidenza) = 0.073       0.165
P(alta  | evidenza) = 0.926       0.831      ← predizione MAP
```

Questo vino ha grado alcolico 12.9 % vol ma acidità volatile pari esattamente a 0.30 g/L: non soddisfa **`AciditaVolatileBassa`** (< 0.3), quindi nemmeno **`VinoMaturoEPulito`**, e la KB — che ragiona con soglie rigide — lo classifica *media* per default. La rete bayesiana combina invece tutte le evidenze e assegna alla classe corretta una probabilità di 0.926, più alta del Naive Bayes (0.831), che conta in modo indipendente concetti correlati.

```
=== Risultati: Rete bayesiana TAN (OWL, 100% etichettati) ===
Precision macro : 0.528 ± 0.008
Recall macro    : 0.556 ± 0.010
F1 macro        : 0.534 ± 0.008
G-Mean          : 0.539 ± 0.009
```

Valutazione generale e limiti

La rete TAN è il **miglior modello tra quelli che usano solo i concetti** (F1 = 0.534). A parità di etichette (100%) supera il Naive Bayes: F1 = 0.534 contro 0.518, migliore in 9 run su 10. La risposta a **RQ3** è quindi positiva: modellare le dipendenze tra i concetti è utile. Il TAN supera anche il Decision Tree sulle misure grezze (0.522), segno che i 31 concetti binari conservano buona parte dell'informazione utile. I limiti principali sono tre:

- **un solo genitore per concetto**: l'albero non può rappresentare dipendenze da più concetti insieme, come quella di `VinoEquilibrato` dai suoi quattro componenti;
- **perdita di informazione per discretizzazione**: i concetti sono soglie fisse sulle misure continue, e la Random Forest sulle misure grezze resta avanti (0.608);
- **struttura appresa a ogni run**: la struttura può variare leggermente tra i run; quella mostrata è del primo.

Confronto Complessivo e Valutazione Statistica

Tabella riassuntiva — tutti i modelli

La tabella seguente riporta media e deviazione standard di tutte le metriche sui 10 run di StratifiedShuffleSplit (test_size = 30%):

Modello	Feature	Etichette	Precision	Recall	F1	G-Mean
DT	Raw	100%	0.518 ± 0.015	0.555 ± 0.014	0.522 ± 0.015	0.535 ± 0.017
RF	**Raw**	**100%**	**0.618 ± 0.011**	**0.601 ± 0.010**	**0.608 ± 0.010**	**0.595 ± 0.010**
LR	Raw	100%	0.548 ± 0.011	0.594 ± 0.010	0.552 ± 0.012	0.567 ± 0.012
DT+OWL	Raw+OWL	100%	0.519 ± 0.011	0.557 ± 0.011	0.522 ± 0.012	0.537 ± 0.013
RF+OWL	Raw+OWL	100%	0.608 ± 0.009	0.599 ± 0.009	0.603 ± 0.008	0.594 ± 0.009
LR+OWL	Raw+OWL	100%	0.543 ± 0.009	0.590 ± 0.009	0.545 ± 0.009	0.560 ± 0.010
BNB	OWL	20%	0.502 ± 0.009	0.532 ± 0.009	0.505 ± 0.010	0.512 ± 0.010
BNB-EM	OWL	20% + non etich.	0.431 ± 0.008	0.468 ± 0.009	0.424 ± 0.008	0.446 ± 0.009
TAN	OWL	100%	0.528 ± 0.008	0.556 ± 0.010	0.534 ± 0.008	0.539 ± 0.009
KB	OWL + regole	nessuna	0.501 ± 0.012	0.439 ± 0.010	0.441 ± 0.010	0.376 ± 0.010

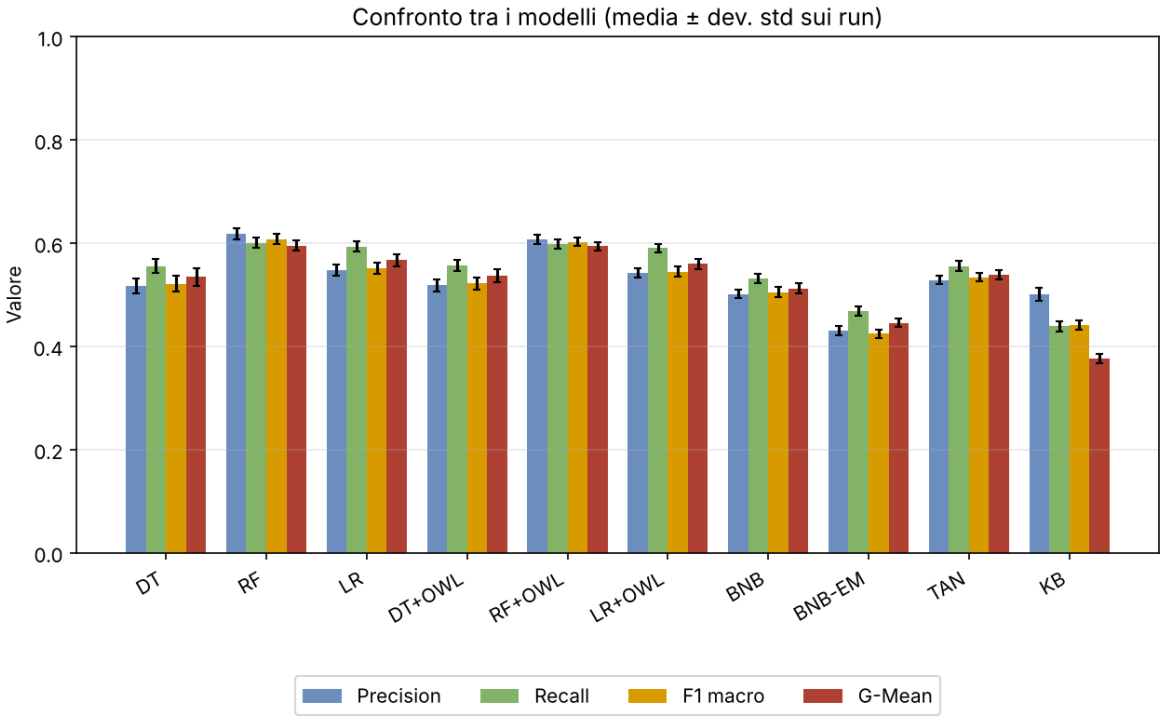


Figura 6 — Confronto delle metriche (Precision, Recall, F1 macro, G-Mean): media ± deviazione standard sui 10 run per ciascuno dei 10 modelli.

Test di Friedman e Nemenyi

Per verificare se le differenze tra i modelli siano statisticamente significative, è stato applicato il **test di Friedman** sulla F1 macro dei 10 modelli nei 10 run (i run sono i blocchi), come raccomandato da Demšar (2006) per il confronto di più classificatori:

```
Friedman test (F1 macro, 10 modelli × 10 run):
  χ² = 84.63
  p-value = 1.93e-14   (< 0.0001)

→ DIFFERENZE STATISTICAMENTE SIGNIFICATIVE (α = 0.05)

Differenza critica di Nemenyi:
  CD = q_0.05 · √( k(k+1) / (6N) ) = 3.164 · √( 110 / 60 ) = 4.28 ranghi
```

Poiché p < 0.05, è stato applicato il **post-hoc di Nemenyi**: due modelli differiscono significativamente se i loro ranghi medi distano più di CD = 4.28. I ranghi medi (1 = migliore) sono:

Posizione	Modello	Rango medio	Distanza da RF	Diverso da RF? (Nemenyi)
1	RF	1.10	—	—
2	RF+OWL	1.90	0.80	No (p = 1.000)
3	LR	3.40	2.30	No (p = 0.797)
4	LR+OWL	4.20	3.10	No (p = 0.396)
5	TAN	5.00	3.90	No (p = 0.111)
6	DT+OWL	6.20	5.10	Sì (p = 0.006)
7	DT	6.40	5.30	Sì (p = 0.004)
8	BNB	7.80	6.70	Sì (p < 0.001)
9	KB	9.10	8.00	Sì (p < 0.001)
10	BNB-EM	9.90	8.80	Sì (p < 0.001)

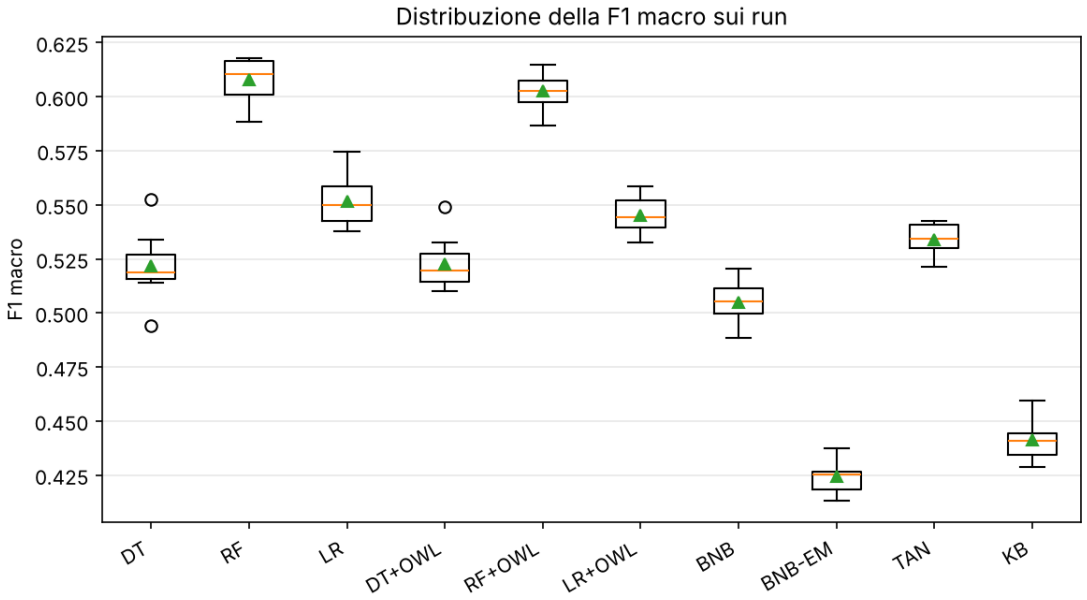


Figura 7 — Boxplot della distribuzione della F1 macro sui 10 run per ogni modello. La linea arancione è la mediana, il triangolo verde la media; i whisker arrivano al valore più estremo entro 1.5 volte lo scarto interquartile, i cerchi sono gli outlier.

I risultati principali del post-hoc di Nemenyi sono:

Confronto	p-value Nemenyi	Significativo?	Interpretazione
RF vs DT	0.004	Sì (p < 0.05)	RF significativamente migliore del singolo albero
RF vs KB	< 0.001	Sì (p < 0.05)	I modelli appresi superano nettamente le sole regole
TAN vs BNB-EM	0.011	Sì (p < 0.05)	Il TAN supera nettamente il BNB-EM
LR vs BNB	0.038	Sì (p < 0.05)	La LR supera il BNB addestrato sul 20% delle etichette
RF vs LR	0.797	No	Differenza non significativa
RF vs TAN	0.111	No	Differenza non significativa, nonostante 0.07 di F1

Confronto	p-value Nemenyi	Significativo?	Interpretazione
RF vs RF+OWL	1.000	No	Praticamente equivalenti: la BK non cambia la RF
LR vs LR+OWL	1.000	No	Praticamente equivalenti
DT vs DT+OWL	1.000	No	Identici
BNB vs BNB-EM	0.871	No	Non significativa, benché EM peggiori in tutti i run
BNB vs KB	0.994	No	La KB senza etichette è alla pari del BNB

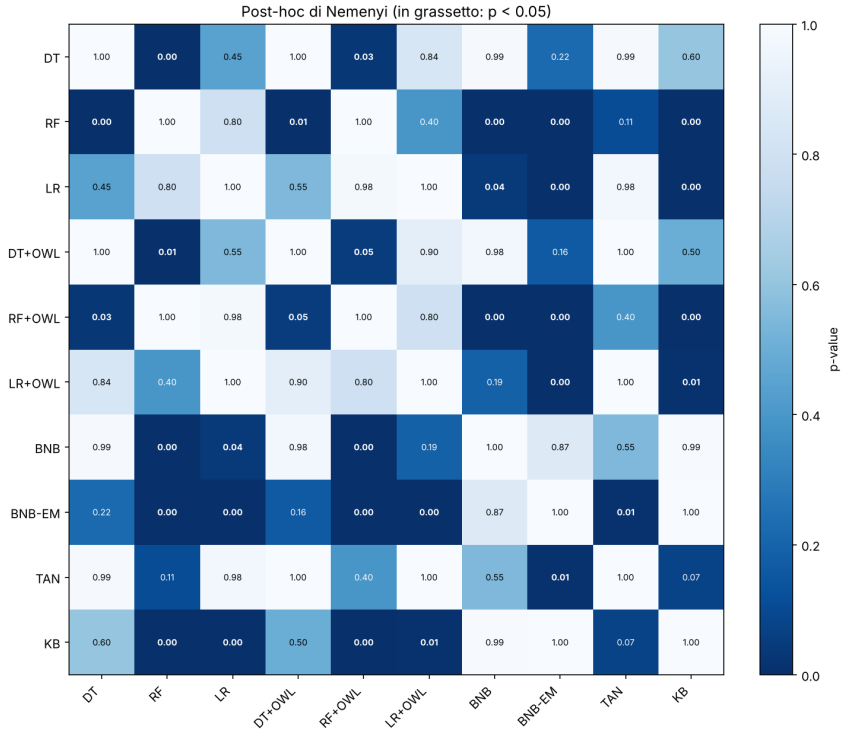


Figura 8 — Heatmap dei p-value del test post-hoc di Nemenyi (F1 macro). Celle scure e valori in grassetto: $p < 0.05$, differenza statisticamente significativa.

Nota Metodologica: Con 10 modelli e soli 10 run il test di Nemenyi è conservativo (CD = 4.28 ranghi): differenze molto regolari possono non risultare significative. Ad esempio il BNB-EM è peggiore del BNB in 10 run su 10, ma la differenza dei ranghi medi (2.1) resta sotto la soglia. Le conclusioni vanno quindi lette insieme alle tabelle per run e ai boxplot.

Matrici di confusione

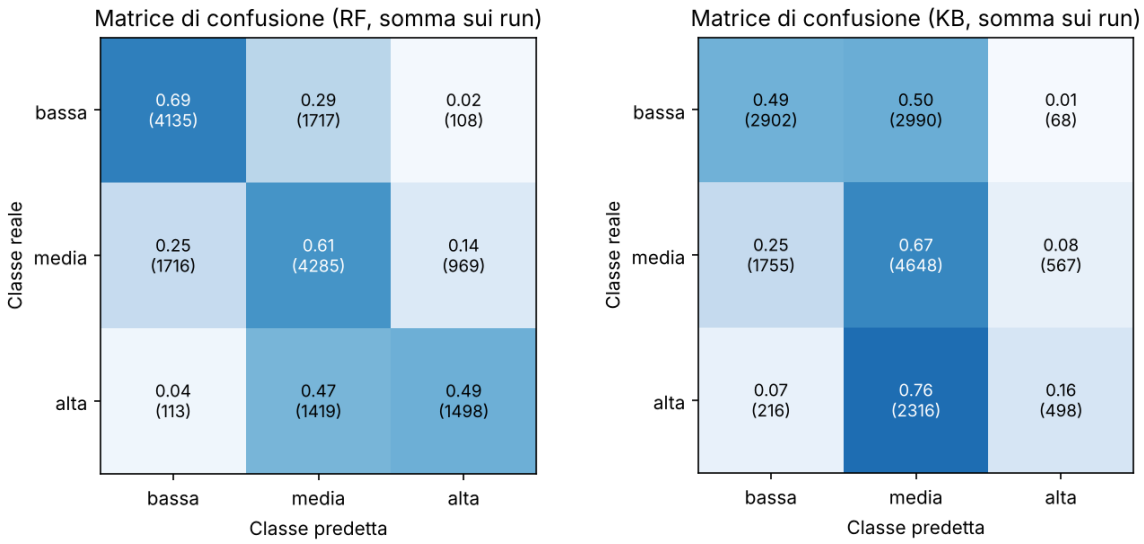


Figura 9 — Matrici di confusione normalizzate per riga (somma sui 10 run): Random Forest, il modello migliore (a sinistra), e Knowledge Base Prolog (a destra). Tra parentesi il numero di vini.

La matrice della Random Forest mostra che gli errori avvengono quasi sempre tra **classi adiacenti**: solo l'1.8% dei vini di qualità bassa è scambiato per alta e il 3.7% dei vini di qualità alta per bassa. È la struttura ordinale del problema: la classe *media* fa da cuscinetto e assorbe la maggior parte degli errori. La KB riconosce bene la classe *media* (recall 0.67) e discretamente la *bassa* (0.49), ma solo il 16.4% dei vini di qualità alta: le regole per la classe *alta* sono troppo restrittive e il 76.4% di questi vini finisce nella classe di default. È questo che porta la G-Mean della KB a 0.376, il valore più basso tra tutti i modelli, pur con una precision (0.501) pari a quella del BNB (0.502).

Analisi dell'impatto della Background Knowledge

Il confronto sistematico tra i modelli addestrati su Raw e su Raw+OWL rivela un quadro chiaro:

Modello	ΔF1 (raw → +OWL)	ΔG-Mean (raw → +OWL)	Run con +OWL migliore	Conclusione
Decision Tree	+0.001	+0.002	5/10	BK neutra: l'albero trova da solo le stesse soglie
Random Forest	−0.005	−0.001	1/10	BK leggermente dannosa: diluisce le feature candidate
Logistic Regression	−0.006	−0.007	2/10	BK leggermente dannosa: più varianza che informazione

La BK OWL **non migliora** i modelli supervisionati che vedono già le misure grezze (risposta negativa a **RQ1**), e nessuna delle tre differenze è significativa (Nemenyi $p > 0.99$). Il motivo è strutturale: i concetti sono funzioni deterministiche a soglia delle stesse misure, quindi non portano informazione nuova ma solo una sua discretizzazione, e i modelli flessibili come la RF apprendono soglie migliori direttamente dai dati. Il valore della Background Knowledge emerge invece nei modelli che la usano come **rappresentazione**: la rete TAN sui soli 31 concetti (F1 = 0.534) supera il Decision Tree sulle misure grezze, il BNB raggiunge F1 = 0.464 con l'1% delle etichette, e la KB classifica senza alcuna etichetta spiegando ogni decisione (risposta a **RQ4**: F1 = 0.441 contro 0.505 del BNB, differenza non significativa, ma con recall molto bassa sulla classe *alta*).

Conclusioni

Nel presente progetto è stato sviluppato un sistema integrato per la classificazione della qualità del vino, affrontato da prospettive differenti e complementari. L'obiettivo non è stato esclusivamente quello di ottenere buone prestazioni predittive, ma di costruire un sistema che integri apprendimento automatico, conoscenza ontologica formale e ragionamento simbolico in modo metodologicamente fondato, con un'unica fonte della conoscenza da cui derivano in modo coerente ontologia, Knowledge Base e feature.

Background Knowledge OWL: l'ontologia `wine.owl`, con 52 classi e 34 concetti enologici definiti, si è dimostrata uno strumento efficace di formalizzazione della conoscenza di dominio, verificato da test automatici di coerenza con Python e Prolog. Il suo impatto come feature aggiuntiva per i modelli supervisionati è trascurabile (ΔF1 tra −0.006 e +0.001, mai significativo), perché i concetti sono soglie sulle stesse misure; il suo valore sta nell'interpretabilità e nella compattezza della rappresentazione.

Apprendimento supervisionato: la Random Forest su feature grezze è il modello migliore, con F1 = 0.608 ± 0.010 e G-Mean = 0.595, primo per rango medio (1.10); il test di Friedman ($\chi^2 = 84.63$, $p = 1.9e\text{-}14$) conferma differenze significative tra i modelli, e il post-hoc di Nemenyi indica che la RF è significativamente migliore di DT, DT+OWL, BNB, BNB-EM e KB. La classe *media* resta la più difficile per tutti i modelli; la classe minoritaria *alta* è gestita al meglio dalla Logistic Regression grazie ai pesi bilanciati.

BNB e BNB-EM: il Bernoulli Naive Bayes sui soli concetti, con il 20% delle etichette, ottiene F1 = 0.505. L'EM peggiora il modello (F1 = 0.424) perché i cluster che massimizzano la verosimiglianza seguono la separazione tra rossi e bianchi e non la qualità; ridurre il peso dei non etichettati ($\lambda = 0.1$) attenua il problema. È un esempio concreto del rischio dei dati non etichettati quando le ipotesi del modello generativo sono violate.

Knowledge Base Prolog: senza vedere alcuna etichetta, le 4 regole dell'esperto raggiungono F1 = 0.441 e precision = 0.501, statisticamente alla pari del BNB, e ogni decisione è accompagnata dalla regola che l'ha prodotta. Le regole per la classe *bassa* sono precise; quelle per la classe *alta* sono troppo restrittive e rappresentano il principale margine di miglioramento.

Rete Bayesiana: il Tree-Augmented Naive Bayes è il miglior modello sui soli concetti (F1 = 0.534) e supera il Naive Bayes a parità di etichette in 9 run su 10. La struttura appresa ritrova le dipendenze introdotte dall'ontologia (famiglie di concetti esclusivi, concetti composti) e relazioni chimiche reali, mostrando che la modellazione esplicita delle dipendenze è più fedele al dominio dell'ipotesi di indipendenza del Naive Bayes.

Sviluppi futuri

- **Soglie per tipologia**: distinguere le soglie dei concetti per rossi e bianchi e documentarle con fonti puntuali (regolamenti UE, OIV, manuali di enologia).
- **Modellare il tipo di vino** nei modelli generativi, stimando P(x | classe, tipo), così che l'EM non scambi la separazione tra rossi e bianchi per la qualità.
- **Qualità ordinale**: sfruttare l'ordine tra le classi con modelli e metriche ordinali (ad esempio regressione logistica ordinale ed errori pesati per distanza tra classi).
- **Reasoner OWL**: integrare un reasoner (HermiT o Pellet) con gli individui (ABox) per classificare i vini direttamente nell'ontologia e interrogarla con SPARQL.
- **Regole raffinate dai dati**: estendere la KB con regole indotte o calibrate sui dati (ad esempio con Inductive Logic Programming), mantenendo la spiegabilità, in particolare per la classe *alta*.
- **Generalizzazione**: applicare il sistema ad altri vini oltre al *vinho verde*, per verificare la portata della Background Knowledge in contesti diversi.

Riproducibilità

I risultati principali (tabella riassuntiva, test statistici, curva semi-supervisionata, regole della KB e grafici) si ottengono con i comandi seguenti; seed e iperparametri sono in **`config.py`**, quindi con lo stesso dataset i numeri sono deterministici:

```
pip install -r requirements.txt
python data/download_dataset.py        # scarica UCI Wine Quality -> data/wine_quality.csv
python main.py                         # 10 run, test statistici, curva, grafici
python main.py --tune --no-curve       # variante con ottimizzazione degli iperparametri
python tests/test_consistency.py       # 10 test di coerenza e correttezza
python analisi_supplementari.py        # analisi di sensibilità (results/*.json)
```

Le analisi di sensibilità (profondità del DT, convergenza OOB, parametri C e α, convergenza e cluster dell'EM, esempio di inferenza e confronto TAN–Naive Bayes) e le matrici di confusione sommate sono calcolate dallo script **`analisi_supplementari.py`**, che usa gli stessi moduli del progetto, gli stessi 10 split e gli stessi seed dell'esperimento principale.

Riferimenti bibliografici

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., Reis, J. (2009). *Modeling wine preferences by data mining from physicochemical properties*. Decision Support Systems 47(4), 547–553. UCI Machine Learning Repository, doi.org/10.24432/C56S3T.
- Nigam, K., McCallum, A., Thrun, S., Mitchell, T. (2000). *Text classification from labeled and unlabeled documents using EM*. Machine Learning 39(2–3), 103–134.
- Friedman, N., Geiger, D., Goldszmidt, M. (1997). *Bayesian network classifiers*. Machine Learning 29(2–3), 131–163.
- Demšar, J. (2006). *Statistical comparisons of classifiers over multiple data sets*. Journal of Machine Learning Research 7, 1–30.
- Cozman, F., Cohen, I. (2002). *Unlabeled data can degrade classification performance of generative classifiers*. Proceedings of FLAIRS.
- Poole, D., Mackworth, A. (2023). *Artificial Intelligence: Foundations of Computational Agents* (3a ed.). Cambridge University Press.
- W3C (2012). *OWL 2 Web Ontology Language: Document Overview* (Second Edition).